A Closed-Loop Bilevel Framework with Routing-in-the-Loop Evaluation for Flexible Job Shop Scheduling and Conflict-Free AGV Routing

QI YU, School of Cyber Science and Engineering, Southeast University, PR China

WEI LI*, School of Computer Science and Engineering, Southeast University, PR China

Integrated scheduling of processing and transport in flexible job shops usually treats travel times as a location-indexed constant matrix, i.e. it assumes that vehicles do not interfere with one another. When a finite fleet shares a sparse corridor network, travel time is no longer a fixed datum but the outcome of routing decisions (in this paper, routing is conflict-free path planning of vehicles on the corridor network); one job's occupancy of a corridor affects the operation times of other jobs. We study the following problem: each operation can be processed by one of several robotic arms that are homogeneous in capability and heterogeneous in efficiency, while AGVs must travel conflict-free on that network. Machine flexibility turns this coupling into an operational trade-off: when the corridor to a fast arm is congested, a slower arm with better connectivity may be the better assignment. Prior work has not measured this trade-off within the evaluation objective of the search.

We propose a *closed-loop bilevel framework*: the upper level decides machine assignment and operation sequencing, the lower level generates conflict-free paths by time-window reservation, and the two levels are connected by an evaluation path and a decision path. The evaluation path is *routing-in-the-loop evaluation*: the fitness of each candidate is the makespan after conflict-free lower-level routing, so yielding wait enters the objective of the search instead of serving only as a post-hoc check. The decision path uses *reservation-table-aware dispatching*: a vehicle is selected after a trial query of both transport legs on the reservation table, rather than by unimpeded travel time. To keep each evaluation *low-overhead*, we give two optimizations—admissible lower-bound pruning and winner-path reuse—and prove that they change neither the selected vehicle nor the final timetable.

Experiments use 10 controlled instances (same size, contention intensity from 6.6% to 33.5%), 10 random seeds per instance, an equal wall-clock budget, and paired Wilcoxon signed-rank tests (p-values Holm-corrected by family, with Hodges–Lehmann pseudomedians and 95% confidence intervals), against a four-level baseline ladder. Relative to the open loop (the search ignores conflicts), the closed loop (the search counts conflict-induced delay) reduces makespan by 18.86% ($p < 10^{-4}$, pseudomedian 18.28%, CI [14.68%, 21.95%]); within the closed loop, replacing rule-based dispatching with reservation-table-aware dispatching reduces it by a further 2.45% ($p = 0.0054$, CI [0.94%, 5.19%]); the complete method reduces makespan by 21.35% relative to the existing research baseline (94 wins, 5 losses, $p < 10^{-4}$), close to the value 22.6% predicted if the two factors combined independently.

The two mechanisms differ markedly in effect size and robustness. The closed loop is the main effect and remains significant per instance on 9 instances after correction; reservation-table-aware dispatching is a much smaller secondary effect, significant only when all instances are pooled and on none individually. The two are near-additive; no positive interaction is detected.

Additional Key Words and Phrases: flexible job shop scheduling, conflict-free routing, closed-loop bilevel framework, routing-in-the-loop evaluation, reservation-table-aware vehicle dispatching

*corresponding author

Authors' Contact Information: Qi Yu, 230240012@seu.edu.cn, School of Cyber Science and Engineering, Southeast University, Nanjing, Jiangsu, PR China; Wei Li, xchlw@seu.edu.cn, School of Computer Science and Engineering, Southeast University, Nanjing, Jiangsu, PR China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

ACM Reference Format:
Qi Yu and Wei Li. 2026. A Closed-Loop Bilevel Framework with Routing-in-the-Loop Evaluation for Flexible Job Shop Scheduling and Conflict-Free AGV Routing. 1, 1 (September 2026), 46 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

1.1 Background and motivation

Flexible job shop scheduling with transportation has long rested on a convenient assumption: the time a job needs to move between two machines is a constant, read from a matrix indexed by location pairs. This is the dominant convention in the literature [22], and recent models still adopt it widely [4, 14, 16]. The assumption is reasonable when the fleet is small enough relative to the corridor network that vehicles rarely meet. Real shops are not built that way: a few vehicles share a sparse corridor network, and a single-lane segment cannot be traversed in both directions at once; once fleet density is high enough, the fleet's own traffic turns from a secondary disturbance into the main cause of delay [9]. Travel time is then no longer a fixed datum but the outcome of two decisions: the path a vehicle takes (this paper calls such conflict-free path planning routing), and the order in which contended segments are reserved. One job's occupancy of a corridor therefore affects the operation times of other jobs; this delay stems from contention, not from the distance between two points.

Such coupled delay among jobs leaves room for optimization only if the scheduler can adjust the assignment, and machine flexibility is what allows an operation to be reassigned to another arm (i.e. to change its processing location). When an operation may be processed on any robotic arm in an option set, and the processing time depends on the chosen arm, the assignment decision selects both the destination and the processing duration. Suppose an operation may be given either to a fast arm at the far end of the shop or to a slower arm next to the load/unload point. Under a constant travel-time matrix the comparison is arithmetic: a shorter processing time plus a longer travel time, against a longer processing time plus a shorter travel time. With corridor contention it is different, because the corridor to the fast arm may be the one every other vehicle is using, and the wait that arises there is invisible to the constant matrix. A slower arm with better connectivity may then be the better choice; and on the same instance, at a different congestion level, the conclusion may differ again. The trade-off requires both ingredients at once: heterogeneous optional arms give the scheduler a destination it can change, and distinct paths on the corridor network make the vehicles' corridor contention different.

Figure 1 shows, on a recomputable minimal instance, that the same pair of assignments ranks the fast arm better under a constant travel-time matrix and the slow arm better under conflict-free routing. The instance has 4 jobs and 2 AGVs: the focal job J1 has one operation that may use M1 ($t^P = 3$) or M2 ($t^P = 14$); three background jobs can be processed only on M3; finished jobs return to the load/unload point. The only path to the fast arm M1 is an exclusive trunk, while the slow arm M2 lies on a separate branch; the background jobs cross the trunk repeatedly and keep it occupied.

Figure 1 contains two kinds of quantity. The first is the unimpeded chain of the focal job J1: J1 leaves the load/unload point LU, is processed, and returns to LU, with travel times taken from shortest paths and no other vehicles counted. Assigned to M1 the path is LU $\to v_1 \to v_2 \to$ M1, one way $1 + \tau + 1 = 8$, round trip 16, plus processing 3, hence 19; assigned to M2 it is LU $\to v_1 \to$ M2, one way $1 + 2 = 3$, round trip 6, plus processing 14, hence 20. On the unimpeded matrix the fast arm wins if and only if $19 < 20$, so with $\tau = 6$ and $t^P_{\text{fast}} = 3$ the smallest admissible t^P_{slow} is 14, the minimum heterogeneity at which a reversal can occur. The box at the bottom of Figure 1(a) reports this chain.

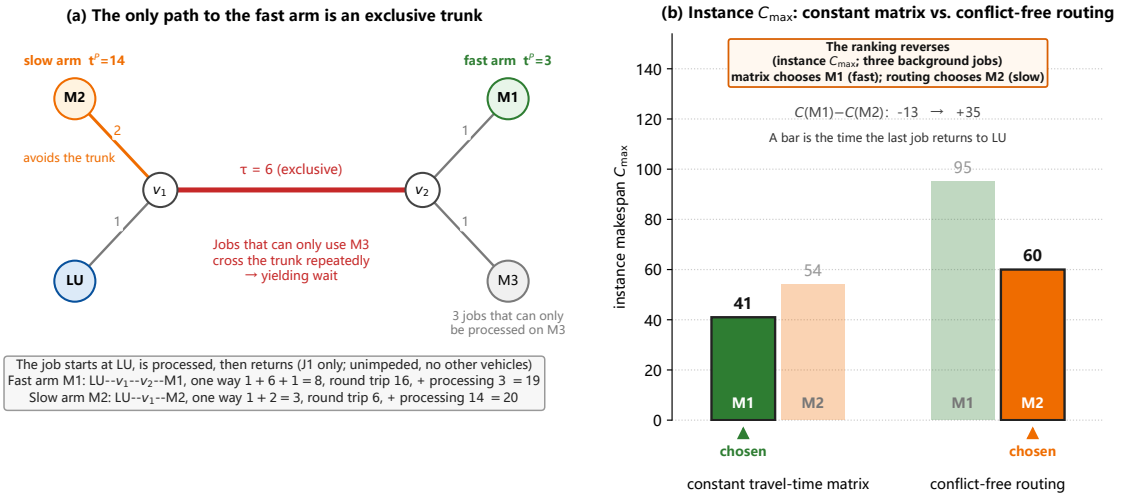


Fig. 1. Motivating instance: the same assignment decision is ranked in opposite order by two evaluations. (a) The only path to the fast arm M1 is an exclusive trunk ($\tau = 6$); the slow arm M2 lies on a short corridor that avoids the trunk; three jobs that can be processed only on M3 cross the trunk repeatedly. The bottom box is the unimpeded chain of the focal job J1 (M1: 19; M2: 20), which sets the minimum heterogeneity required for a reversal. (b) Bar height is the instance makespan $C_{\max}$ (4 jobs, 2 AGVs, same decoder as the main text). Under the constant matrix $C_{\max}$ is 41 versus 54 and the fast arm is chosen; under conflict-free routing the same pair gives 95 versus 60 and the slow arm is chosen, so the difference changes from −13 to +35. This figure is an existence example; of 1200 parameter combinations around this instance, 205 (17%) show the reversal.

The second is the instance makespan $C_{\max}$, the time at which the last finished job returns to LU, obtained from the same decoder as the rest of the paper and shown by the bars in Figure 1(b). All four jobs start from LU and return; J2, J3, and J4 can be processed only on M3 (empty and loaded legs 8 each, processing 3 each); the two AGVs can operate in parallel.

Under the constant matrix, vehicles do not interfere and yielding wait does not enter the evaluation. If J1 is assigned to M1, then $C_{\max} = 41$: in the first wave J1→M1 and J2→M3 run in parallel, each with outbound 8, processing 3, and return 8, and both are back at LU at $t = 19$; in the second wave J3 and J4 both go to M3, leave at $t = 19$, arrive at $t = 27$ after outbound 8, J3 processes 27–30 and returns at 38, and J4 waits for M3 until 30, finishes at 33, and returns after 8, so the last return is 41. If J1 is assigned to M2, then $C_{\max} = 54$: J1 has outbound 3, processing 14, and return 3; AGV1 waits at M2 until $t = 17$ and is back at LU at $t = 20$; J2 still returns at 19. The two vehicles are then no longer synchronized; J4 reaches M3 at 43 and returns to LU at 54. The matrix evaluation still prefers the fast arm.

The same pair of assignments is then evaluated by conflict-free routing. With J1 on M1, $C_{\max} = 95$: J1's unimpeded chain is still 19, but the outbound leg occupies the trunk on $[1, 7)$ and the return on $[12, 18)$, so J2 yields 17 at the trunk entrance and reaches M3 at 25; J2–J4 then serialize on the trunk, and J4 returns to LU at 95. With J1 on M2, $C_{\max} = 60$: J1 takes the branch and does not occupy the trunk; J2 yields only 1 on LU–v_1; the background jobs still yield to one another, and J4 returns to LU at 60. The slow arm is then better. Neither the machine assignment nor the operation sequence has changed; only the evaluation has.

1.2 Gap relative to prior work

Existing work mostly treats either the scheduling of a flexible job shop or the conflict-free travel of vehicles; when the two are combined, the assignment is usually fixed first and routing then resolves the contention, so the resulting delay cannot be fed back to the schedule. For the joint problem of flexible assignment and conflict-free routing, an exact method already solves small instances to proven optimality [21]; we adopt this problem setting, and our contribution lies in the solution method. Beyond the sizes that this exact decomposition can solve, existing methods still split scheduling and conflict-free routing into two successive steps: the first stage fixes machine assignment and operation sequence, and the second stage resolves vehicle conflicts for the frozen schedule [19]. Corridor contention detected in the second stage cannot revise the machines and sequence already chosen in the first. Methods that add feedback only write the transport durations of one execution back onto the time axis, for the current solution only; they neither revise the machine assignment nor repeat this step for every candidate in the search, and their benchmark has a single eligible machine per operation, so reassignment does not arise [10]. Hence, when machine flexibility, heterogeneous processing efficiency, and corridor contention are all present and the schedule targets the makespan with yielding wait counted, this trade-off has not yet been measured within the evaluation objective of the search. Section 2 positions this paper in the literature.

1.3 Approach

To address the gap that contention is not fed back to the schedule and that the search does not count yielding wait, we adopt a bilevel structure: the upper level decides machine assignment and operation sequencing, and the lower level generates conflict-free paths by time-window reservation; the two levels are connected by an evaluation path and a decision path, and the contribution of each path to makespan is measured separately.

The *evaluation path* takes the fitness of each candidate as the makespan after the lower level has completed conflict-free routing, so yielding wait enters the objective that the search optimizes rather than a post-hoc feasibility check. One conflict-free decode is more than an order of magnitude more expensive than a decode on the unimpeded shortest-path matrix; under a fixed compute budget, a higher cost per evaluation reduces the number of evaluations that can be completed. [10] lists “extending this mechanism into a fully iterative loop” as future work, but does not report its computational cost or its effect on makespan. We measure this trade-off under an equal wall-clock budget.

The *decision path* uses *reservation-table-aware dispatching*. One transport consists of two legs, empty pickup and loaded delivery. For each candidate vehicle, the lower-level router runs a trial query on the current reservation table for the earliest feasible times of the two legs in turn (the trial query does not write the final timetable), and chooses the vehicle with the earliest feasible delivery, rather than the vehicle whose arrival is earliest by the unimpeded-shortest-path estimate. This rule replaces the idle-first dispatching used by [10]. To keep each evaluation *low-overhead* under an equal wall-clock budget, this paper gives two optimizations that do not change the output: admissible lower-bound pruning, and winner-path reuse.

Two further inter-layer interfaces are also implemented: first, an extra cost on occupied corridor–time slots, so that later paths avoid those slots; second, changing the processing machine or adjusting the operation sequence according to the corridor on which a yield occurred and the transport that occupied that corridor first. Under an equal wall-clock budget neither yields a stable makespan gain; both are reported as controls in Section 5.

1.4 Main results

To measure the contributions of closed-loop evaluation and of reservation-table-aware dispatching separately, we construct a four-level baseline ladder whose first two levels correspond to the structures of two published works: **B0** is open-loop planning plus actual execution, corresponding to [10]; **B0+** replaces only the vehicle assignment on a frozen schedule with reservation-table-aware dispatching, corresponding to the two-stage structure of [19]; **B1** is closed-loop planning plus rule-based dispatching; **B2** is our method. The four configurations form a 2×2 design along two factors: whether the search counts conflict-induced delay, and whether a vehicle is selected by unimpeded time or by a trial query on the reservation table. Only a comparison that changes a single factor can attribute a gain to the corresponding mechanism. The makespan that an open-loop plan reports for itself is often infeasible on the actual corridors, so every schedule is executed by the same conflict-free router and all comparisons use the realized makespan after execution.

Both factors yield a significant gain. With the dispatching rule fixed, including yielding wait in the fitness reduces makespan by 18.86% ($p < 10^{-4}$), the contribution of closed-loop evaluation; within the closed loop, replacing rule-based dispatching with reservation-table-aware dispatching reduces it by a further 2.45% ($p = 0.0054$), the contribution of the dispatching mechanism; end to end, relative to the literature structure, the reduction is 21.35% (94 wins, 5 losses, $p < 10^{-4}$). The two low-overhead optimizations speed up a dispatching evaluation by a factor of 2.33, cut router calls by 47.4%, and lower the cost multiple from 12.2 to 5.23; this is the precondition for dispatching to yield a net gain under an equal wall-clock budget.

Both mechanisms are significant, but they are *not substitutes for each other*: reservation-table-aware dispatching under the open loop yields 4.59% ($p < 10^{-4}$), far less than the closed-loop 18.86%; the end-to-end 21.35% is close to the value 22.6% predicted under independence, i.e. near-additive, with no positive interaction detected. A single contrast from the open loop to the complete method would merge the few percentage points from dispatching into the closed-loop gain, so we report the two factors separately.

Results grouped by layout, vehicle-to-arm ratio, and flexibility are in Section 5.5. Per-instance tests are Holm-corrected by family; without correction, the probability of at least one "significant" result among the 10 instances would be far above the nominal level. After correction, the closed-loop gain remains significant on 9/10 instances and rises roughly with contention intensity, so it can be interpreted along the layout axis; reservation-table-aware dispatching is significant on none of the instances (2 before correction), and across the four vehicle-to-arm levels the gain changes sign and never reaches significance. Dispatching is therefore significant only when instances are pooled; with 10 seeds per instance, the sample cannot yet determine on which class of instances it is stably positive.

These conclusions rest on two comparison rules (Section 5.1.2). First, the cost per evaluation can differ by about two orders of magnitude across methods; comparing at an equal number of generations would give more compute to the more expensive method, so we align on wall-clock time. Second, each contrast changes only one factor (whether the search counts delay, or how a vehicle is selected); a contrast that changes both yields only a combined effect and cannot separate the closed-loop gain from the dispatching gain.

1.5 Contributions

(1) **Instances: a controllable characterization of the reassignment trade-off.** When machine flexibility, heterogeneous processing efficiency, and corridor contention are all present and the schedule targets the makespan with yielding wait counted, a slower arm with better connectivity may be the better assignment if the corridor to a fast arm is congested. Models that schedule on a constant travel-time matrix cannot capture this comparison, because

the matrix contains no yielding wait [19]; models with conflict-free routing but a fixed machine per operation do not involve it either, because reassignment does not arise [10]. On top of the existing problem setting [21], we construct instance families with controllable contention intensity, used to examine how high contention must be before counting yielding wait in the schedule yields a gain.

(2) **Framework: a closed-loop bilevel framework, with a measurement of the contribution of routing-in-the-loop evaluation to makespan.** *Routing-in-the-loop evaluation* takes the fitness of each candidate as the makespan after conflict-free lower-level routing, so yielding wait enters the objective of the search. We measure the computational cost and the benefit of this practice under an equal wall-clock budget: relative to the open loop (the search ignores conflicts), the closed loop reduces makespan by 18.86% ($p < 10^{-4}$) under rule-based dispatching and by 17.75% ($p < 10^{-4}$) under reservation-table-aware dispatching.

(3) **Mechanism: reservation-table-aware dispatching, and two optimizations that keep it *low-overhead*.** Within the closed loop, replacing rule-based dispatching with reservation-table-aware dispatching reduces makespan by a further 2.45% ($p = 0.0054$). We give two optimizations, admissible lower-bound pruning and winner-path reuse, and prove that they change neither the selected vehicle nor the final timetable; router calls fall by 47.4%, the cost multiple per evaluation falls from 12.2 to 5.23, and the speedup is 2.33.

(4) **Evidence: the two paths differ in magnitude, are near-additive, and are not substitutes for each other.** The closed loop remains significant after correction on 9/10 instances; the gain of reservation-table-aware dispatching is much smaller and significant only when instances are pooled; the end-to-end gain is close to the prediction under independent composition of the two factors. Changing only vehicle selection under the open loop is far from a substitute for counting yielding wait in the search. Two further inter-layer interfaces (a surcharge on occupied corridor–time slots, and reassignment or resequencing from the yield record) are reported as controls in Section 5; under an equal wall-clock budget neither yields a stable gain.

The notation of the paper is collected in Table 2.

2 Related work

Related work falls into two strands. One is flexible job shop scheduling with transportation times: it decides what is processed where and in what order; vehicles enter only as capacity resources, and travel times are taken from a constant matrix. The other is conflict-free multi-robot routing: it resolves vehicle contention in space and time, but does not change the start, end, or release time of a task. We review the two strands in turn, then the work that combines them in one problem, and finally state the gap addressed here.

2.1 Flexible job shop scheduling with transportation

The flexible job shop scheduling problem with processing machines and transport vehicles (FJSP_PT) extends FJSP by bringing material movement into the schedule. Xin et al. [22] systematically review progress in the field through 2023, organize its assumptions, constraints, objectives, and benchmark instances, and divide solution methods into exact, heuristic, metaheuristic, and swarm-intelligence classes. Modelling, constraints, and solution methods of the general flexible job shop are covered by a contemporaneous review [1]. The public benchmarks of this family are collected by Homayouni and Fontes [7]. The dominant modelling convention among them is a constant travel-time matrix indexed by location pairs: a vehicle needs a fixed $t(a, b)$ to move a job from a to b, which does not change with whether other

vehicles occupy the same corridor. Vehicles therefore enter the model only as capacity resources: a finite fleet competes for transport tasks, but corridor occupancy intervals are unconstrained.

Under this modelling convention, solution methods have advanced quickly. Meng et al. [14] give a constraint-programming model of FJSP with multiple AGVs, close 29 previously open benchmark instances and improve 27 best-known solutions, and couple it with a dual-population genetic algorithm assisted by constraint programming. Qin et al. [16] combine a quality–diversity archive with Q-learning region selection, and design a local search jointly guided by AGVs and the critical path, but the critical path is still computed on matrix travel times. Fu et al. [4] use an improved genetic algorithm to decide production and logistics jointly in a flexible assembly shop; "integration" there means that the two classes of decision are solved together, not that transport duration varies with the schedule. In all of these works transport durations are taken from the matrix, so no vehicle is delayed by another through corridor contention.

These assumptions, however, "reduce the complexity of the problem and even change its nature" [22]. For example, when many transport vehicles are in use, the time at which a job arrives at a machine is in practice affected by traffic congestion; only when the dependence between the processing and transport stages is handled can the resulting plan be feasible on the actual corridors.

This strand has machine flexibility but schedules on a constant travel-time matrix, and can therefore optimize machine assignment and operation sequencing thoroughly. The matrix contains no yielding wait: two transports never delay each other through contention, and the ranking of an assignment does not change with corridor occupancy. This paper keeps its flexible modelling and its metaheuristic solution framework, and only replaces the constant transport duration in the matrix with the traversal time after conflict-free routing—which may contain yielding wait caused by other vehicles.

2.2 Conflict-free routing with exogenous tasks and congestion-aware dispatching

The other strand takes corridor contention itself as the object. Multi-agent path finding (MAPF) requires several agents to walk from start to goal without collision on a shared graph; optimal solution is NP-hard [5]; for a single vehicle finding a path on an already occupied graph, recent work attaches Safe Interval Path Planning to two-dimensional road networks and continuous time [8]. Especially in settings with continuously arriving tasks, congestion already reduces the number of tasks completed per unit time when the network is far from saturation. Zhang et al. [25] optimize a road network with edge weights, use the weights to steer agents away from bottlenecks and reduce head-on meetings, and thereby let several solvers of continuously arriving tasks complete more tasks. In a manufacturing setting, Kim and Kim [9] give a congestion function, a congestion penalty, and congestion-aware dispatching rules for a large AGV fleet, and evaluate them in discrete-event simulation. A survey of multi-AGV management lists dispatching as a layer independent of path planning, and idle-first and nearest-vehicle as common single-attribute rules among them [12]; a survey of planning and control of autonomous mobile robots lists task allocation and path planning as adjacent layers on the same control stack [3].

Two points of this strand are directly relevant to this paper. First, the release time, start, and end of a task are inputs to the router, which cannot revise the machine assignment that produced those tasks. Second, congestion is usually quantified not by the yielding wait that actually occurred on a transport but by an indirect measure: [9] uses local vehicle density, and [25] uses offline-learned edge weights of the road network (the larger the weight, the less a vehicle is encouraged to take that edge). Density or an edge weight can only show that a corridor has relatively many vehicles; a yield can name which corridor, which interval, and which transport occupied it first.

This strand has conflict-free routing but no machine flexibility: tasks are given externally, and assignment lies outside its decision scope. We keep its conflict-free path generation and let the upper-level schedule adjust the assignment and operation sequence according to the yields that occur in routing.

2.3 Integrating scheduling and conflict-free routing: feedback depth and our position

Work that writes flexible assignment and conflict-free routing into one problem is scarce. van Os and Basten [21] formalize FJSP with conflict-free transportation constraints: an operation has optional machines and a processing time that varies with location, and vehicles must travel conflict-free. The solver uses logic-based Benders decomposition—a constraint-programming master gives machine assignment and operation sequence, and an integer-programming subproblem plans conflict-free paths for that schedule; if a path is infeasible, the subproblem writes the conflict as a new constraint and sends it back to the master, forbidding a schedule of the same kind. Most benchmark instances are solved to proven optimality, and about half of them improve makespan by at least 10% relative to prior heuristics. The method is limited by the instance size on which the exact decomposition can prove optimality. The authors note that making fuller use of the information "which conflicts caused the infeasibility" is a natural follow-up. We adopt this problem setting and, on sizes that the exact decomposition cannot handle, use a heuristic to feed routing delay back into the scheduling decision.

Once the instance size exceeds that exact decomposition, existing heuristics still split scheduling and conflict-free routing into two successive steps. Sun et al. [19] first fix machine assignment and operation sequence by a hybrid genetic algorithm, then resolve conflicts by a genetic algorithm with an AGV encoding and time-window Dijkstra; contention detected in the second stage cannot revise the machines and sequence already chosen in the first. Li et al. [10] add one feedback step: the path layer writes the realized transport durations back onto the time axis, only shifts start times, does not revise the machine assignment, and acts on the current solution only; their benchmark has a single eligible machine per operation, so reassignment does not arise.

The gap in existing work is not whether vehicle conflicts are handled, but how the routing result enters the schedule. Sun et al. [19] and Li et al. [10] both include conflict-free travel in their pipelines; they differ in the frequency and content of feedback: whether every candidate is evaluated by actual routing or feedback is given once for the current solution, and whether only transport durations are written back or the schedule can revise the machine assignment accordingly.

Critical-chain reasoning is a mature device in job-shop local search [15, 20], and recent FJSP work with transportation times uses it as well [16]. If the critical chain is computed on a constant travel-time matrix, however, a start delay can be traced only to a predecessor operation or a busy machine, not to a specific corridor, because the matrix contains no corridor occupancy. With conflict-free routing by time-window reservation, the same delay can be located on a specific segment, a specific interval, and the transport that occupied that segment first.

On instance design, [22] notes that instances in the field "differ mainly in size and lack a dedicated design for the values of key problem parameters", and in particular lists the ratio of mean transport time to mean processing time $\bar{T}_t/\bar{T}_p$ as a quantity that is highly likely to affect algorithm performance, and calls for an expanded test set. On top of this, this paper supplements a parameter that is visible only after routing is modelled explicitly: how much of an instance's congestion is assignment-related. Congestion concentrated at the load/unload exit is crossed by every job under every assignment and therefore does not change with any decision; congestion on a corridor that serves only some machines can be bypassed by reassignment. Two instances with the same $\bar{T}_t/\bar{T}_p$ may leave very different room

Table 1. Comparison of related work with this paper. "Machine heterogeneity" means that an operation has several optional machines and the processing time varies with the machine; "coupling frequency" means the number and direction of times the routing result enters the schedule; the last column is the content fed back to the schedule. We also implement two further interfaces, a surcharge on corridor–time slots and the yield record, reported as controls in Section 5.

Work class	Ref.	Machine het.	Vehicle contention	Coupling freq.	Feedback content
FJSP_PT metaheuristics	[4, 14, 16]	yes	constant matrix	—	—
Lifelong MAPF	[5, 25]	—	conflict-free	tasks exogenous	edge weights
Congestion-aware dispatching	[9]	—	density proxy	one-way	congestion penalty
Two-stage FJSP + routing	[19]	yes	conflict-free	one-way	none
RL scheduling + PBS	[10]	no	conflict-free	one-shot correction	arrival times
Exact LBBD	[21]	yes	conflict-free	master–subproblem	conflict-driven cuts
This paper		yes	conflict-free	every candidate	realized time + trial query

for the scheduling mechanism. This paper therefore generates instances whose contention intensity is controllable (Section 5).

Table 1 summarizes the comparison. At sizes that the exact decomposition cannot handle, scheduling and routing are either unconnected, connected schedule-then-resolve, or connected by a single feedback step on the current solution. At these sizes we give a bilevel framework: every candidate is evaluated by the makespan after conflict-free routing, and reservation-table-aware dispatching selects a vehicle by a trial query; two optimizations reduce the cost per evaluation without changing the selected vehicle or the timetable. The contributions of the two paths are measured separately.

3 Problem statement and model

3.1 Problem statement

An instance consists of n jobs, N_M robotic arms, and N_A vehicles (AGVs), operating on a shop-floor road network represented as a graph. Job j has a fixed chain of n_j operations $O_{j1} \to \cdots \to O_{jn_j}$; operation O_{ji} may be processed on any robotic arm in its option set $\Omega_{ji} \subseteq M$, taking time t^P_{jim} on arm m, a value that depends on the chosen arm. Robotic arms occupy fixed nodes of the graph; jobs are moved between these nodes by the fleet. Solving an instance means determining four groups of decisions at once:

(i) assign each operation O_{ji} to some robotic arm $m \in \Omega_{ji}$;
(ii) sequence the operations assigned to each robotic arm;
(iii) assign the induced transport tasks to vehicles and sequence the tasks of each vehicle;
(iv) plan a path on the graph for every vehicle move, requiring temporal and spatial conflict-freedom and allowing waits.

The objective is to minimize the makespan $C_{\max}$.

3.2 Modelling assumptions

A1. Static instance and complete information. All n jobs are already at v_0 at time $t = 0$; there are no release dates or due dates. Processing data, the layout G, and the vehicle states are known at planning time.

A2. Jobs. Each job has a fixed precedence chain. Processing and transport are non-preemptive; jobs cannot be split; there are no exogenous precedence constraints among different jobs.

A3. Robotic arms: partial flexibility and heterogeneous processing times. Each operation O_{ji} may be processed on any $m \in \Omega_{ji}$; most operations satisfy $|\Omega_{ji}| \geq 2$, and full flexibility $\Omega_{ji} = M$ is a special case. Processing times

Table 2. Notation.

Symbol	Meaning
$J = \{1, \ldots, n\}$, O_{ji}	set of jobs; the i-th operation of job j, $i \le n_j$
$M = \{1, \ldots, N_M\}$, $\Omega_{ji} \subseteq M$	set of robotic arms; option set of O_{ji}
t^P_{jim}	processing time of O_{ji} on arm $m \in \Omega_{ji}$
$K = \{1, \ldots, N_A\}$	homogeneous unit-load vehicles
$G = (V, E)$, τ_e	shop-floor graph; traversal time of corridor $e \in E$
v_0, $\mathrm{loc}(m)$	load/unload point; node occupied by arm m
$t^*(a, b)$	shortest-path time from a to b in G, ignoring contention
$\delta_{\mathrm{ret}} \in \{0, 1\}$	whether a finished job returns to v_0 (baseline value 1)
S_{ji}, C_{ji}, A_{ji}	start, completion, and material-arrival times of O_{ji}
$C_{\max}$	makespan

depend on the chosen arm, and the arms need not share a uniform ranking by speed: when $m \neq m'$, t^P_{jim} and $t^P_{jim'}$ need not be proportional. An arm processes only one operation at a time, does not fail during the planning horizon, and sequence-dependent setup is folded into t^P_{jim}.

A4. Buffers and loading. The input and output buffers of each arm, and the storage at v_0, have ample capacity, so blocking does not occur. Loading and unloading times are negligible and do not occupy the arm.

A5. Vehicles. The N_A vehicles are homogeneous and unit-load; they travel at constant speed whether empty or loaded; acceleration, deceleration, and battery charge are not modelled. All vehicles stand empty at v_0 at $t = 0$ and are immediately available. A transport task is a one-way move—from v_0 to the first arm, between two successive distinct arms, or, when $\delta_{\mathrm{ret}} = 1$, from the last arm back to v_0; any vehicle may execute any task, so successive moves of the same job may be relayed across different vehicles. After unloading, a vehicle is not forced back to v_0; it repositions according to its own task chain, and its departure time may be postponed by the routing layer.

A6. Road network and conflict-freedom. G is strongly connected; the traversal time τ_e of each corridor is a known constant. Corridors are *exclusive* resources: the two directions of e share a single reservation, so at most one vehicle occupies e at any time, which rules out both head-on conflicts and same-direction overtaking. Occupancy intervals are half-open, $[t,\ t + \tau_e)$, so a second vehicle may enter exactly at $t + \tau_e$. Nodes have ample berths, independent of the traversal resource; waits therefore occur at berths and are always permitted.

A7. Out of scope. Machine and vehicle failures, finite buffers and blocking, battery charging, and dynamic job arrivals [6, 23] are left to future work (item (5) of Section 6.3).

Of these, A1, A2, A4, and A5 are standard assumptions in the FJSP_PT literature [22]. The features that define our problem are A3 (machine-dependent processing times on partially flexible robotic arms) and A6 (contention stated as an explicit exclusive-resource constraint rather than a constant travel-time matrix); as a result, transport duration is an output of the routing decision, not a fixed input parameter. Deadlock-freedom is not an assumption: Proposition 2 in Section 4.3 uses A4 and A6 to prove that any assignment and operation sequence can be decoded into a conflict-free, deadlock-free timetable, so the search need not repair infeasible solutions.

3.3 Decision variables and induced transport tasks

Let $x_{jim} \in \{0, 1\}$ indicate that O_{ji} is assigned to robotic arm m; write $\mathrm{m}(j, i)$ for the chosen arm and $d_{ji} = \mathrm{loc}(\mathrm{m}(j, i))$ for the node of that arm. When $\delta_{\mathrm{ret}} = 1$, a zero-duration *dummy operation* O_{j,n_j+1} at v_0 is appended to each job,

representing the return to the load/unload point after completion; in this way return moves and intermediate moves share the same task notation, and the makespan is always the completion time of each job's last operation.

Unlike a vehicle routing problem, transport tasks are not part of the input; they are *induced* by the assignment:

$$\mathcal{T}(x) = \Big\{ (j,i) \;:\; d_{j,i-1} \neq d_{ji} \Big\}, \qquad d_{j0} \equiv v_0, \tag{1}$$

where task (j,i) moves job j from $d_{j,i-1}$ to d_{ji} and becomes ready at time $r_{ji} = C_{j,i-1}$ (with the convention $C_{j0} = 0$). If two successive operations are placed on the same arm, no transport task is created. Thus both the *number* and the *endpoints* of transport tasks are consequences of decision group (i): an assignment that keeps a job on the same arm throughout needs only two moves, whereas an assignment that changes arm at every operation needs $n_j + 1$ moves. Assignment of tasks to vehicles is indicated by $y_{tk} \in \{0,1\}$, with $\sum_{k\in K} y_{tk} = 1$ for each $t \in \mathcal{T}(x)$; tasks given to the same vehicle are ordered into a sequence.

In a constant travel-time matrix model, the assignment likewise determines the number of moves and their endpoints, but the duration of each move is a constant in the matrix, and corridor occupancy does not enter the model. In this paper, the same assignment also determines on which corridors those moves contend with one another.

3.4 Timing relations and feasibility

Write $p_{ji} = \sum_{m\in\Omega_{ji}} x_{jim} t^{P}_{jim}$ for the processing time on the chosen arm. Assignment, precedence, and the unit-capacity of each arm give

$$\sum_{m\in\Omega_{ji}} x_{jim} = 1, \qquad \forall j,i, \tag{2}$$

$$C_{ji} = S_{ji} + p_{ji}, \qquad \forall j,i, \tag{3}$$

$$S_{ji} \;\geq\; A_{ji}, \qquad \forall j,i, \tag{4}$$

$$S_{j'i'} \;\geq\; C_{ji} \;\;\vee\;\; S_{ji} \;\geq\; C_{j'i'} \qquad \forall (j,i) \neq (j',i') \text{ on the same arm}, \tag{5}$$

where A_{ji} is the time at which the job actually appears at d_{ji}. If $(j,i) \notin \mathcal{T}(x)$, the job is already there and $A_{ji} = C_{j,i-1}$. Otherwise A_{ji} is produced by the vehicle that carries task (j,i): write a^{E}_{ji} for the time at which that vehicle reaches the pickup node after its empty leg, and set

$$\ell_{ji} = \max\big(a^{\mathrm{E}}_{ji},\; r_{ji}\big), \qquad A_{ji} = \text{arrival time of the loaded leg departing at } \ell_{ji}, \tag{6}$$

i.e. the vehicle may wait for the job, and the job may wait for the vehicle. Whether empty or loaded, each leg is a walk $(e_1,\dots,e_q)$ in G whose corridor entry times $\eta_1,\dots,\eta_q$ satisfy

$$\eta_1 \;\geq\; \sigma, \qquad \eta_{l+1} \;\geq\; \eta_l + \tau_{e_l}, \quad l = 1,\dots,q-1, \tag{7}$$

where σ is the time at which the leg may start. If $\eta_{l+1} = \eta_l + \tau_{e_l}$, the vehicle enters the next corridor without stopping; if $\eta_{l+1} > \eta_l + \tau_{e_l}$, the difference is a wait at a berth. The leg ends at $\eta_q + \tau_{e_q}$. Finally, the corridor-contention constraint is

$$\big[\eta,\; \eta+\tau_e\big) \;\cap\; \big[\eta',\; \eta'+\tau_e\big) \;=\; \emptyset \tag{8}$$

This equation holds for any two distinct occupancies of the same corridor e, independently of direction and of which vehicles are involved. The objective is

$$\min\; C_{\max}, \qquad C_{\max} = \max_{j\in J}\; C_{j,\,n_j+\delta_{\mathrm{ret}}}. \tag{9}$$

Two corollaries are worth recording, because the algorithms of Section 4 rest on them. First, in any semi-active schedule, (4) and (5) tighten to

$$S_{ji} = \max\big(A_{ji},\ F_{\mathrm{m}(j,i)}\big), \tag{10}$$

where F_m is the completion time of the operation sequenced immediately before O_{ji} on arm m. Thus the start time is the later of the job's arrival time and the completion time of the preceding operation on that arm, and later sections use this to attribute a postponed start to waiting for material or waiting for the arm. Second, (8) is the only constraint in the model that couples different jobs through *space* rather than through a shared machine.

3.5 Relation to FJSP_PT

After (8) is removed, legs no longer occupy corridors against one another, so each leg may take a shortest path and need not wait in between. Transport durations collapse to the constant matrix $t^*(a, b)$, and the model becomes flexible job shop scheduling with transportation (FJSP_PT). The two problems therefore differ only in this set of corridor-exclusivity constraints: with them, the duration of a move is jointly determined by the vehicles' paths; without them, the duration is an input parameter. The resulting t^* is used later as a lower bound, and in the open-loop baseline to estimate arrival times by unimpeded travel time.

Relative to work that likewise supplies an explicit road network, the difference lies in what is reserved. van Os and Basten [21] reserve *nodes*: two vehicles may not occupy the same node at the same time, nor swap nodes. This paper reserves *edges*: corridors are exclusive in both directions, and at most one vehicle occupies a corridor at any time. Lyu et al. [13] likewise use a bidirectional network with edge exclusivity, but additionally require that a node hold at most one vehicle at a time; this paper does not impose that node-capacity constraint, and waits occur at ample berths. Why the instances cannot be compared directly is explained under limitation (1) of Section 6.2.

3.6 Computational complexity

Proposition 1. *The problem defined by (2)–(9) is strongly NP-hard; it remains so when every operation satisfies* $\Omega_{ji} = M$.

Proof. Restrict to instances in which $\tau_e = 0$ for every $e \in E$, each robotic arm is connected to v_0 by a dedicated corridor, and $N_A = n$. Every leg then has duration zero and occupies the empty half-open interval $[t, t) = \emptyset$, so (8) holds identically and no vehicle can delay another; moreover, because each job has a dedicated vehicle, (6) never binds, and $A_{ji} = C_{j,i-1}$. What remains is exactly (2)–(5) with $A_{ji} = C_{j,i-1}$, i.e. the flexible job shop scheduling problem with unrelated machines. Two special cases of that problem are already strongly NP-hard: taking $|\Omega_{ji}| = 1$ everywhere reduces it to the classical job shop, which is strongly NP-hard already with three machines; taking $\Omega_{ji} = M$ and $n_j = 1$ reduces it to minimizing makespan on unrelated parallel machines, which contains identical-parallel-machine scheduling and is strongly NP-hard when the number of machines is part of the input. Flexible job shop scheduling in general is itself strongly NP-hard [1]. Each of the restrictions above is a polynomial reduction, so the general problem is strongly NP-hard; the second special case also shows that full flexibility does not make the problem easy. □

The reduction of Proposition 1 sets corridor traversal times to zero and thereby excludes vehicle contention. This shows that the problem is already strongly NP-hard even without corridor contention. After decisions (i)–(iii) are further fixed, the remaining path planning is still a conflict-free multi-agent routing problem [5], and the release times and endpoints of the tasks are determined by the operation chains and the assignment, so they constrain one another. Conflict-free routing therefore adds constraints on top of the scheduling hardness; it does not replace it. Exact methods

already exist at small scale: van Os and Basten [21] use logic-based Benders decomposition and solve most benchmarks under the same problem setting to proven optimality. This paper targets sizes that the exact decomposition cannot handle.

3.7 Instance features

This paper holds instance size fixed and varies only structure; the effect of size itself is discussed under limitation (1) of Section 6.2. Each instance records the following features. The first four are quantities common in the scheduling literature; Xin et al. [22] note that $\bar{T}_t/\bar{T}_p$ in particular is likely to affect algorithm performance.

- $\bar{T}_t/\bar{T}_p$: the mean shortest-path travel time among arm nodes divided by the mean processing time, i.e. the ratio of transport time to processing time.
- Heterogeneity H: the coefficient of variation of each operation's processing times over its optional arms, then averaged over all operations (operations with $|\Omega_{ji}| = 1$ contribute 0). $H = 0$ means that optional arms have identical processing times: reassignment does not affect processing time but can still be used to avoid congestion. As H grows, processing-time differences increasingly dominate the assignment and the room for reassignment to avoid congestion shrinks, so we do not expect the gain to rise with H; the measurements grouped by H in Section 5.5 agree with this.
- Flexibility $F = \mathrm{mean}_{ji}\, |\Omega_{ji}|/N_M$: the mean number of optional arms per operation as a fraction of all arms, i.e. the scope of reassignment.
- N_A/N_M: the vehicle-to-arm ratio. A smaller ratio makes the bottleneck more likely to lie in transport; a larger ratio, in processing.

We also record two quantities that are meaningful only on an explicit road network, describing the severity of congestion and the part of it that reassignment can avoid. Let $\mathrm{cut}(v_0)$ be a smallest set of corridors whose deletion disconnects the load/unload point v_0 from every robotic arm.

- Contention intensity: on the initial random population, the share of total travel time that is yielding wait. It is determined by the instance structure, not supplied as an input; we use it to order instances, and the per-instance values are in Table 4.
- $|\mathrm{cut}(v_0)|$ (funnel width): an upper bound on the number of vehicles that can leave the load/unload point at the same time. The value 1 means that every vehicle must pass through the same corridor to leave. The layout family (Family A) varies contention by adding or removing such parallel exits; relative to `high`, `funnel` halves this value.

Contention intensity cannot separate avoidable from unavoidable congestion. For this purpose we defined the *funnel share* $\varphi \in [0, 1]$, the fraction of a typical $v_0 \to$arm trip spent on corridors shared by *every* shortest $v_0 \to$arm path; on our instances, however, two layouts that should differ by construction give almost the same φ, so it does not enter the later analysis, and a better measure is left to future work (item (1) of Section 6.3).

Each instance carries a composite lower bound $\mathrm{LB} = \max(\mathrm{LB}_{\mathrm{chain}}, \mathrm{LB}_{\mathrm{load}}, \mathrm{LB}_{\mathrm{cut}})$, computed directly from the instance data. The three components each relax a different part of the model; none is always the largest on every instance. $\mathrm{LB}_{\mathrm{chain}}$ takes, for each job, "the shortest inbound trip + $\sum_i \min_{m\in\Omega_{ji}} t^P_{jim}$ + the shortest return trip", i.e. it ignores inter-arm moves and all queues. $\mathrm{LB}_{\mathrm{load}}$ takes $\sum_{j,i} \min_m t^P_{jim}/N_M$, i.e. it ignores operation precedence and transport. $\mathrm{LB}_{\mathrm{cut}}$ uses the funnel: when $\delta_{\mathrm{ret}} = 1$ each job crosses $\mathrm{cut}(v_0)$ twice; spreading $X = 2n$ crossings over the corridors of that cut so as to minimize the load on the busiest one yields

$$C_{\max} \;\geq\; X \Big/ \sum_{e\in\mathrm{cut}(v_0)} \tau_e^{-1}. \tag{11}$$

On the instances of this paper, the largest of the three is LB_{cut} (eight of ten instances) or LB_{chain} (the other two); LB_{load} is never the largest—which agrees with an instance design that concentrates contention at the load/unload exit. The bound is fairly loose: the gap to the best known solution is roughly between 27% and 58%, so it is used only to check the order of magnitude, not to prove that a solution is near-optimal.

4 Closed-loop bilevel framework

4.1 Overview: routing-in-the-loop evaluation and the decision path

The framework assigns the four groups of decisions of Section 3 to two levels. The input is an instance: the operation chains of the jobs, the option sets and processing times of the arms, the fleet size, and the corridor network. The upper level searches over assignment and sequencing with a memetic algorithm (MA) [24]; an event-driven decoder in the middle translates an upper-level candidate into a sequence of precedence-constrained transport tasks; the lower level turns that sequence into conflict-free paths by time-window reservation. The output is an executable conflict-free timetable and its makespan. Relative to two-stage schemes or schemes that correct only once, the difference lies not in having two levels but in how the routing result enters the schedule: the content and direction of feedback, and whether every candidate receives it.

The evaluation path, i.e. *routing-in-the-loop evaluation.* In every generation, the fitness of each candidate is the makespan after the lower level has completed conflict-free routing of all transport tasks, yielding wait included. The upper level optimizes this value rather than the makespan computed from unimpeded shortest paths.

The decision path. Vehicle selection reads the current reservation table instead of learning the outcome only after paths have been generated. The mechanism is *reservation-table-aware dispatching*: for each candidate vehicle, the earliest feasible times of the empty and loaded legs are obtained by a trial query on the reservation table (the trial query does not write the final timetable), and the vehicle with the earliest feasible delivery is selected, rather than the vehicle whose arrival is earliest by the unimpeded-shortest-path estimate.

The contributions of the two paths to makespan are measured separately in Section 5. The two further inter-layer interfaces implemented are a surcharge on occupied corridor–time slots (hereafter, a surcharge) and reassignment or resequencing from the yield record; their constructions are in Sections 4.4.1 and 4.6, and the control results in Section 5.

The structure of the framework is shown in Figure 2. The two paths enter the upper level at different places, and the computational cost of a single evaluation therefore differs as well.

4.2 Lower level: time-window reservation and conflict-free routing

4.2.1 Reservation table. The only contended resource is the corridor (assumption A6), so the reservation table maps each corridor to a time-ordered list of half-open occupancy intervals $[t,\ t + \tau_e)$, each labelled with the vehicle and task that occupy it. The table supports four operations: query the earliest feasible entry time no earlier than a given instant; write an occupancy; revoke every occupancy of a given task; and save the current table state and restore it after a trial query (checkpoint and rollback). Nodes are not reserved: traversing a node takes zero time, and waits occur at berths (assumption A6 already separates berths from the traversal resource). Only one resource—the corridor—therefore needs to be reserved, and both the lower-level conflict-free path planner (hereafter, the router) and a checker independent of the decoder inspect only corridor occupancy.

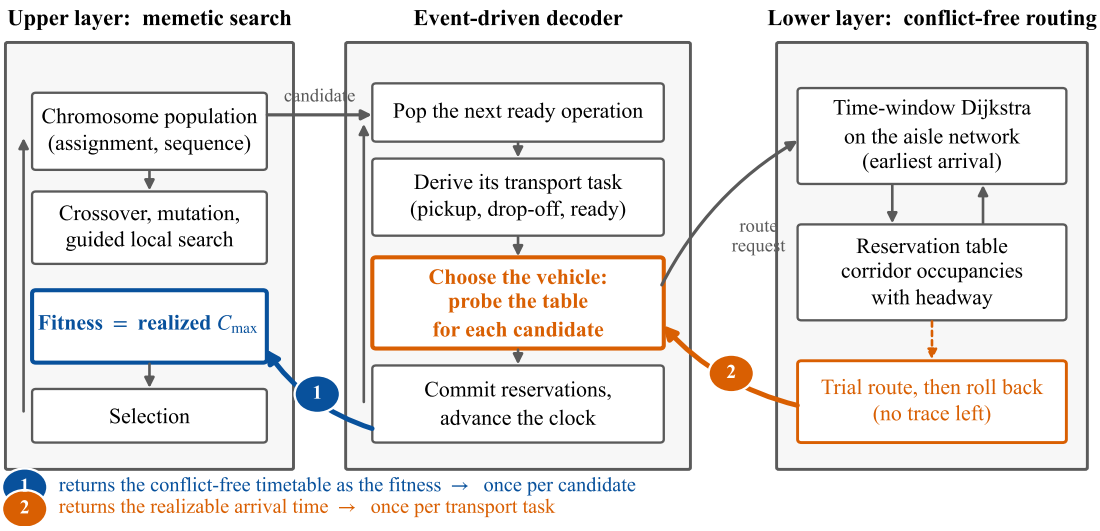


Fig. 2. The closed-loop bilevel framework. The upper level searches over assignment and sequencing with a memetic algorithm; the lower level performs conflict-free routing by time-window reservation; an event-driven decoder in the middle translates an upper-level candidate into a sequence of transport tasks. The two feedback paths are distinguished by colour and enter the upper level at different places: path 1 (blue, marked 1) returns the completed conflict-free timetable as fitness, once per candidate; path 2 (orange, marked 2) runs a two-leg trial query on the reservation table for the earliest feasible delivery time before a vehicle is chosen, once per transport task; the trial query does not write the final timetable. The four-level baseline ladder of Section 5 is arranged by whether these two paths are open.

4.2.2 Time-window Dijkstra. A leg is planned by a label-setting search. A label (u, t) means "standing at the berth of node u at time t", and the cost is t (Algorithm 1). When a corridor leaving the current node is examined, the reservation table is queried for the earliest feasible entry time $t' \geq t$; the gap $t' - t$ is the time the vehicle waits at the berth of u. The wait in (7) is therefore produced directly by the search and need not be modelled separately. Within one decoding, paths already written to the reservation table are not revoked, so vehicles are planned in the fixed order in which the decoder generates tasks, and a later-planned vehicle yields to an earlier-planned one.

Lemma 1 (Earliest arrival is sufficient). *Keeping only the earliest-arrival label at each node is sufficient to obtain a path that reaches the destination earliest under the current reservation table.*

Proof. Berths have ample capacity and do not occupy the traversal resource (A6), so a vehicle that has stood at u since time t need only wait further until t' in order to take any continuation that a vehicle arriving at u only at $t' > t$ could take. An earlier-arrival label is therefore never worse, and each node need keep only the earliest arrival. Moreover, the later the query time to the reservation table, the fewer the still-feasible entry slots, so the time to enter corridor e and finish traversing it, $t \mapsto R.\textsc{EarliestEntry}(e, t, \tau_e) + \tau_e$, is non-decreasing in t. The search extracts labels in increasing order of arrival time; the first time the destination is extracted is the earliest arrival. □

Each node therefore needs only one earliest-arrival label, and ordinary Dijkstra suffices; there is no need for the per-interval labelling of Safe Interval Path Planning (SIPP) [8]. If a second criterion is introduced (for example, an extra cost on busy corridor–time slots), earlier arrival is no longer always better, and each node must keep several

Algorithm 1 ROUTE: conflict-free planning of a single leg

Require: origin s, destination g, earliest departure t_0, reservation table R, write-to-table flag *commit*
Ensure: a sequence of arrival time and corridor occupancy intervals; if *commit* is true, those intervals are also written to R

place the label (s, t_0) in a priority queue keyed by (time, node id) ▷ ties broken by node id, so the order is deterministic
while the queue is nonempty **do**
  extract the label (u, t) of least key
  if $u = g$ **then**
    backtrack to obtain the occupancy intervals; if *commit* is true, write them to R one by one; **return** arrival time t
  end if
  for all corridors $e = (u, v)$ leaving u **do**
    $t_{\text{in}} \leftarrow R.\text{EARLIESTENTRY}(e, t, \tau_e)$ ▷ wait at the berth of u until t_{in}
    if $t_{\text{in}} + \tau_e$ is strictly earlier than the incumbent arrival at v, update the label $(v,\ t_{\text{in}} + \tau_e)$
  end for
end while
return failure ▷ should not occur when G is strongly connected; see the text

mutually non-dominated labels. The resulting increase in cost per evaluation is the main reason why a surcharge on corridor–time slots does not yield a stable makespan gain under an equal wall-clock budget (Section 4.4.1).

The number of reservations is finite, time is bounded, and G is strongly connected, so a path always exists: in the worst case the vehicle waits until every existing occupancy has ended and then travels along an unimpeded shortest path. This guarantees Proposition 2 below, so the search need not repair infeasible solutions. A single call costs $O\big(|E|\,(W^2 + \log|V|)\big)$: at most $|E|$ corridors are examined; for each, $O(W)$ candidate entry times are enumerated, each candidate is tested for overlap in $O(W)$ time, and a heap operation of $O(\log|V|)$ is performed, where W is the mean number of reserved intervals per corridor. If the reservation table were replaced by an ordered structure supporting a logarithmic earliest-entry query, the complexity would drop to $O\big(|E|(\log W + \log|V|)\big)$; the implementation in this paper uses the linear scan above.

Running the same search on an empty reservation table over all pickup–delivery node pairs yields the unimpeded shortest-path matrix t^* of Section 3. It is used in three places: estimating arrival times by unimpeded time in open-loop configurations, the proximity term in the reassignment neighbourhood, and the travel times of the two-stage baseline; it is also the admissible lower bound used by the pruning of Section 4.5.

4.3 Upper level: encoding and event-driven decoding

4.3.1 Chromosome encoding. Let the total number of operations be N. An individual consists of two segments: a *machine-assignment vector* of length N, indexed by a fixed enumeration, whose k-th component is the robotic arm chosen for the k-th operation from Ω_{ji}; and an *operation sequence*, a permutation that allows a job to appear more than once—job j appears n_j times, and the x-th appearance stands for O_{jx}. The sequence supplies a global order: the relative order of operations that share an arm is read from it.

When $\delta_{\text{ret}} = 1$, each job appears once more in the sequence, corresponding to the dummy operation of Section 3 (location fixed at v_0, processing time zero). Return transport therefore uses the same assignment and sequencing as intermediate transport, and the decoder needs no extra branch.

Which vehicle is given a task is not written into the chromosome; it is decided at decode time (Section 4.4). A third segment for dispatching would enlarge the search space and would no longer guarantee that every chromosome is decodable, so a repair operator would be required.

Algorithm 2 DECODE: fitness from a complete conflict-free simulation

Require: instance, road network, assignment x, operation sequence
Ensure: makespan, timetable, per-corridor yield statistics

for every arm set $F_m \leftarrow 0$; for every job set $\text{pos}_j \leftarrow v_0$, $\text{ready}_j \leftarrow 0$
for every vehicle set $\text{loc}_k \leftarrow v_0$, $\text{avail}_k \leftarrow 0$; clear the reservation table
for all operations O_{ji} taken in sequence order (dummy operations included) **do**
  $m \leftarrow$ the arm of O_{ji} under x; $d \leftarrow \text{loc}(m)$
  if $\text{pos}_j = d$ **then**
    $A_{ji} \leftarrow \text{ready}_j$ ▷ successive operations on the same arm; no transport task
  else
    choose vehicle k (Section 4.4)
    $a^{\text{E}} \leftarrow \textsc{Route}(\text{loc}_k, \text{pos}_j, \text{avail}_k, R, \text{true})$; $\ell \leftarrow \max(a^{\text{E}}, \text{ready}_j)$ ▷ the empty leg occupies corridors as well
    $A_{ji} \leftarrow \textsc{Route}(\text{pos}_j, d, \ell, R, \text{true})$; $\text{loc}_k \leftarrow d$; $\text{avail}_k \leftarrow A_{ji}$
    accumulate yielding wait by corridor
  end if
  if $F_m > A_{ji}$ then `bind` $\leftarrow$ `machine`, else `bind` $\leftarrow$ `arrive`
  $S_{ji} \leftarrow \max(A_{ji}, F_m)$; $C_{ji} \leftarrow S_{ji} + p_{ji}$; $F_m \leftarrow C_{ji}$; $\text{pos}_j \leftarrow d$; $\text{ready}_j \leftarrow C_{ji}$
end for
return $C_{\max} = \max_{j,i} C_{ji}$, the timetable, and the per-corridor yield statistics

4.3.2 Event-driven decoder. The decoder (Algorithm 2) implements the evaluation path. It sweeps the operation sequence from left to right, maintaining the idle time of each robotic arm, the current node and ready time of each job, and the current node and available time of each vehicle. When the arm of an operation differs from the job's current location, a transport task of (1) is generated, a vehicle is chosen, and two paths are planned against the *live* reservation table: an empty leg to the pickup and a loaded leg to the destination, joined by (6). The operation then starts by (10); the decoder also records which of the two arguments is larger, as a `bind` field, which later attribution of a postponed start takes as authoritative.

Two properties follow. First, reservations are written in the order in which the decoder generates tasks; ties in time are broken by a fixed rule at both sites—the routing label queue is keyed by (arrival time, node id) and a label is updated only when the new arrival is strictly earlier; dispatching scans vehicles in increasing id order and keeps the first strictly better one (Section 4.5). A chromosome therefore always corresponds to a unique timetable, and evaluation is deterministic. The seed-paired, equal-budget comparisons of Section 5 depend on this. Second, decoding only advances: if a neighbour changes only the part of the sequence after position x, then events and reservations before x remain unchanged. This is the basis of incremental evaluation; the present implementation does not yet use that speed-up.

PROPOSITION 2 (EVERY CHROMOSOME IS DECODABLE). *For any machine assignment and any operation sequence, Algorithm 2 terminates and returns a conflict-free, deadlock-free timetable of finite makespan. The search therefore need not invoke a repair operator.*

PROOF. By the encoding, the operation sequence is a topological order of each job's precedence constraints: the x-th appearance of a job stands for its x-th operation, so ready_j is already known when it is read. Ample buffers (A4) mean that an operation is never blocked for lack of storage, so once the job has arrived, (10) always supplies a finite start time. Each ROUTE call returns in finite time by the argument after Algorithm 1, and occupies only a still-free slot on the reservation table, so (8) holds by construction. Deadlock-freedom is likewise by construction: the corridor is the only contended resource (A6); waits occur only at ample berths and do not occupy corridors; every occupancy written to the table ends at a finite time, so there is no circular wait among vehicles; in the worst case a vehicle waits until every

existing occupancy has ended and then proceeds. The sweep executes $N + n\delta_{\text{ret}}$ times, and each time schedules a finite number of events at finite times. □

4.4 Reservation-table-aware dispatching

A vehicle may be chosen either by an unimpeded-shortest-path estimate or by a trial query on the reservation table. The open-loop rule scores each vehicle by

$$\text{score}(k) = \max\big(\text{avail}_k + t^*(\text{loc}_k, \text{pickup}),\ \text{ready}_j\big) + t^*(\text{pickup}, \text{destination}), \tag{12}$$

which does not consult the reservation table and may therefore select a vehicle that is geometrically near but whose corridor is already occupied. The formula is the same as the idle-first dispatching of [10], and is a typical single-attribute rule in surveys of multi-AGV dispatching [12].

The closed-loop default is instead *reservation-table-aware dispatching*. For each candidate vehicle, the empty leg is planned and temporarily written to the reservation table, the loaded leg is then planned, the earliest feasible delivery time from this trial query is recorded, and the table is rolled back to its state before the trial query. The empty leg must be written temporarily first; otherwise the loaded leg cannot see that occupancy, the two legs may be placed in the same corridor–time slot, and the trial result would be too early. The vehicle with the earliest feasible delivery is selected, not the vehicle that (12) estimates as best.

Vehicles have identical capability (assumption A5) and differ only in current location and corridor occupancy, so contention on the reservation table can directly determine which vehicle is selected.

The cost of the mechanism is that the trial query raises the routing calls per transport task from 2 to at most $2N_A$, and a single evaluation costs about 12.2 times as much as under rule-based dispatching. Unless this overhead is reduced, the extra computation offsets the makespan gain of the mechanism under an equal wall-clock budget. The two speed-ups are in Section 4.5; Section 5 therefore compares the two dispatching rules only under the same wall-clock limit.

4.4.1 Three choices for the inter-layer interface. Once the bilevel structure is fixed, one must choose what the lower level returns to scheduling. We implement the following three interfaces and measure each under an equal wall-clock budget; only the first (realized durations, read by both the evaluation path and the dispatching trial query) yields a stable makespan gain. The other two are constructed below, with results in Section 5.

Return realized durations only. The lower level returns traversal times after conflict-free routing. The evaluation path computes fitness from them, and reservation-table-aware dispatching reads the same quantity during a trial query. This ranks every candidate by the makespan with yielding wait included, but does not indicate which decision caused the wait. Methods that schedule first and route once lack this property: they obtain a realized duration only after the assignment is fixed. The main gain of this paper comes from this interface: every candidate compared by the search has a fitness that already includes the yielding wait caused by that candidate itself.

Return a surcharge on corridor–time slots. By analogy with coordination in Lagrangian relaxation and column generation, a price can be put on the contended resource: attach a shadow price $\pi(e, b)$ to each corridor–time slot, let path planning minimize "arrival time $+\theta\cdot$cost" on the Pareto front ($\theta \geq 0$ is the only tunable parameter), and let reassignment pay that cost for the traversal it needs. We implement two estimates: perturbing the capacity of a single slot and re-solving, and a low-cost approximation refreshed each generation. Under an equal wall-clock budget, makespan consistently worsens.

Mechanistically, a surcharge provides no new information. Wait caused by contention already enters the vehicle's arrival time, then enters makespan through (10), and is counted by the fitness; attaching a further cost to the same occupancy counts the same delay twice. Later tasks not yet planned may be blocked by reservations written earlier, but that effect is likewise already reflected in the makespan after a complete decode. Hence, as long as fitness is taken from a complete conflict-free decode, a congestion cost on the lower-level objective adds no information. We verify this inference only on this one interface.

The main cost of a surcharge follows from Lemma 1: once price is a second criterion, earlier arrival is no longer always better, and single-label Dijkstra must be replaced by a multi-label Pareto search, so the cost per evaluation rises whether or not the price is non-zero. Measurements in Section 5 show that the loss in makespan under an equal wall-clock budget comes mainly from this overhead, not from worse path choices induced by the price.

Return the yield record. The third choice is to return structure. When a yield occurs, path planning already knows the corridor, the time slot, and the transport that first occupied that corridor; the decoder keeps these yield events, not only the total wait. A record is a triple (corridor, time slot, preceding transport), used to decide which operations local search should change. It does not enter any objective: the lower level still minimizes arrival time, the upper level still minimizes makespan; only the generation of the neighbourhood changes. There is therefore no double-counting of the same delay. The operators are in Section 4.6. Under an equal wall-clock budget it likewise has no stable gain: the neighbourhood is well targeted, but its improvement rate is too low to offset the extra decodes consumed.

4.5 Low overhead: admissible lower-bound pruning and winner-path reuse

The mechanism of the previous subsection faces the following problem under an equal wall-clock budget. At an equal number of *generations*, reservation-table-aware dispatching outperforms rule-based dispatching by 2.90% (Figure 7); but a single evaluation costs 12.2 times as much, so at equal *wall-clock time* the gain is offset by the extra computation and the net effect is close to zero. Hence, if the cost multiple can be reduced without changing any selection, the gain of the mechanism becomes visible. This subsection gives two such optimizations, both *provably output-preserving*.

Admissible lower-bound pruning. Conflict-free routing can only be *later* because of yielding, never faster than the unimpeded shortest path that ignores contention. The estimate (12) used by the open-loop rule is therefore an *admissible lower bound* on the realized delivery time:

$$\mathrm{lb}(k) = \max\big(\mathrm{avail}_k + t^*(\mathrm{loc}_k, \mathrm{pickup}),\ \mathrm{ready}_j\big) + t^*(\mathrm{pickup}, \mathrm{destination}) \ \le\ \mathrm{est}(k), \tag{13}$$

where $\mathrm{est}(k)$ is the earliest feasible delivery time obtained by a complete two-leg trial query for vehicle k. Hence if a vehicle's lower bound is already no earlier than the current best trial-query delivery time, routing need not be called for it.

Winner-path reuse. After the trial query the reservation table is rolled back as a whole, the winner's two legs included, and the decoder then recomputes them, causing two redundant routing calls. Since Route($\cdot$, *commit* = true) only writes the legs into the reservation table in order, the paths computed for the winner during the trial query can be cached and committed directly without replanning.

Proposition 3 (The two speed-ups do not change the selected vehicle or the timetable). *In the implementation, vehicles are queried in increasing id order, and the selected vehicle is replaced only when some vehicle's feasible delivery is strictly earlier than the current best. Under this rule, after the pruning of* (13) *and winner-path reuse are added,*

the selected vehicle, the occupancies written to the reservation table, and the final timetable are the same as when a complete two-leg trial query is run for every vehicle.

Proof. Consider pruning first. When vehicle k is reached, let the current best trial-query delivery be est^*. If $\text{lb}(k) \geq \text{est}^*$, then (13) gives $\text{est}(k) \geq \text{lb}(k) \geq \text{est}^*$, so k's feasible delivery cannot be strictly earlier and would not be selected even if every vehicle received a complete trial query; not calling routing for it leaves the selected vehicle unchanged. The reservation table is saved before each trial query and restored afterwards; vehicles for which routing is not called do not rewrite the table and therefore do not affect later queries. Consider reuse next: the two legs of the selected vehicle were computed against the same reservation-table state; after restore, the table is the same state the decoder later writes into. Route$(\cdot, \textit{commit} = \text{true})$ only writes the given legs into the table, so writing the cached paths and replanning then writing produce identical occupancy intervals. □

Together the two optimizations give a 2.33-fold speed-up, cut routing calls by 47.4%, and reduce the cost multiple of a single evaluation from 12.2 to 5.23. These three numbers are the *joint* effect of the two optimizations: the experiment measures them as one on/off switch (Table 8), without attributing them separately to pruning or reuse.

The effect of pruning varies with fleet size. The tighter the bound and the earlier a good trial-query delivery time is found, the more vehicles need no routing call; the larger the fleet, the more candidates, and the total trial-query cost grows approximately linearly with N_A. Measurements in Section 5 show that the pruning effect weakens as the fleet grows but still cuts 42.2% of the routing calls on the largest fleet tested. Together, these effects mean that the gain of the mechanism may vanish beyond some fleet size; Section 5.5 explains why the present number of seeds cannot determine that size.

4.6 Yield-record-guided local search

This subsection writes the third interface of Section 4.4.1 as local-search operators. In each generation, a leading fraction of the population receives local search. Under an equal wall-clock budget the mechanism does not yield a stable makespan gain (Section 5). The attribution along the critical chain in the next subsection is not used only by this operator: the case study of Section 5.8 likewise reads the critical chain by Table 3.

4.6.1 Attribution along the critical chain. Backtracking from the event that determines the makespan yields a chain; but if the chain contains only operations, it still cannot say why a given operation's start was postponed. When `bind` = `arrive`, the operation is waiting for the job to arrive, and the cause may be an unfinished upstream operation, no vehicle available, or corridor contention—three cases that call for different changes. The backtrack therefore includes transport legs as well, and records a type for every link (Table 3). A yield on a loaded leg is always recorded as corridor occupancy, because it directly postpones arrival; on an empty leg the share of yielding wait in that leg's total duration, with threshold 0.5, distinguishes "vehicle" from "corridor".

The typed critical chain provides three kinds of information: the real operations on the chain (from which reassignment draws candidates), the corridors and time slots on the chain, and the transport that first occupied that corridor at that slot and caused the yield. Local search uses the third.

4.6.2 Two families of operators. Corridor contention can be changed in two ways, each corresponding to one operator.

Table 3. Labels of the links on the critical chain, and the changes that can be taken. Only the corridor-occupancy class carries a concrete corridor and time slot, and only it can name the transport that first occupied that corridor.

Label	Criterion	Remedy
`operation`	a real operation is occupying its robotic arm	—
`machine`	`bind` = `machine`, blocked by the preceding operation on that arm	resequence
`upstream`	the vehicle arrived first, the material is not yet ready	recurse to $O_{j,i-1}$
`vehicle`	the material is ready, the vehicle is late, and yielding is a small share of the delay	change vehicle
`corridor`	a yield occurs on a specified corridor at a specified time slot	reassignment or staggering

Reassignment: change the processing location. For at most $L = 5$ real operations on the critical chain, each alternative robotic arm $m' \in \Omega_{ji} \setminus \{m\}$ is scored by

$$\text{score}(m') = t^*\big(\text{pos}_{j,i-1}, \text{loc}(m')\big) + t^P_{jim'}, \tag{14}$$

i.e. the sum of unimpeded travel time and processing time, in the same units; the lowest-scoring arm produces a neighbour that changes only that one assignment. The sum of the two terms expresses the trade-off of this paper: a nearer arm may be slower; when the corridor to a fast arm is occupied, (14) allows a longer processing time to be traded for a shorter path with better traffic conditions. The score contains no cumulative-yield term: such a term is a total over the whole planning horizon and the whole fleet and grows with instance size, whereas the first two terms remain at the scale of a single operation, so the two are not commensurable.

Staggering: change the order. Take the longest corridor-occupancy link on the chain and look up the transport that first occupied that corridor in that slot. This yields two directed neighbours: advance the blocked operation in the sequence so that its transport falls into a less busy slot, or postpone one preceding occupying operation so that the two occupancies no longer overlap. Each round moves only one such operation, keeping the neighbourhood small and its direction clear. A move swaps one appearance of a job with an adjacent appearance of another job; the internal order of that job is unchanged, and the result is still a legal permutation; by Proposition 2, it remains decodable. The neighbourhood therefore has at most $L + 2 = 7$ candidates, each specified by the yield record. A generic mutation samples uniformly at random in the same space.

4.6.3 Acceptance rule. The first neighbour with a smaller makespan is accepted, and the number of rounds is capped (Algorithm 3). Each round *recomputes* the critical chain from the current solution rather than reusing the chain from the start: after a successful change, the chain that determines the makespan often changes, and reusing the old chain would modify a link that no longer determines it.

4.7 Outer genetic algorithm

The outer layer uses a conventional population search. The initial population is generated at random: an arm is drawn uniformly from Ω_{ji}, and the sequence is shuffled; two further individuals are added, one assigning every operation to its fastest arm, one assigning greedily for load balance. Selection is by tournament with a small elite. For crossover, the sequence segment uses order crossover that preserves precedence, and the assignment segment uses uniform crossover; mutation either swaps two positions in the sequence or moves an operation to another optional arm. Fitness is the decoded makespan. Hyperparameters are in Section 5.1.3.

Algorithm 3 ContentionGuidedSearch: yield-record-guided local search

Require: chromosome c and its decode result r
Ensure: a chromosome whose makespan is no worse than $r.C_{\max}$, and its decode result

for *round* = 1 **to** 3 **do**
  $\mathcal{N} \leftarrow \textsc{Reassign}(c, r) \cup \textsc{Stagger}(c, r)$ ▷ at most $L + 2$ neighbours, each specified by the yield record
  improved ← **false**
  for all $c' \in \mathcal{N}$ **do**
    $r' \leftarrow \textsc{Decode}(c')$
    if $r'.C_{\max} < r.C_{\max}$ **then**
      $c \leftarrow c'$; $r \leftarrow r'$; *improved* ← **true**; **break**
    end if
  end for
  if not *improved* **then**
    break
  end if
end for
return c, r

4.8 Complexity and cost

One fitness evaluation processes $N + n\delta_{\text{ret}}$ events. Under closed-loop dispatching each transport uses at most $2N_A$ routing calls, so each individual costs $O\big((N+n)\,N_A\,|E|\,(W^2 + \log|V|)\big)$; open-loop selection by unimpeded time no longer carries the factor N_A. The lower-bound pruning of Section 4.5 does not change this worst-case bound, but by Proposition 3 it reduces the actual number of calls by 47.4% without changing the output, i.e. it replaces N_A by a smaller effective multiple. Local search adds at most $3(L+2)$ further evaluations for each selected individual.

Within the framework, the cost of a single evaluation can differ by about two orders of magnitude. On the ladder batch the measurements are: open-loop 0.38 ms, closed-loop rule-based dispatching 2.70 ms, full closed-loop 14.2 ms, about 37 times the open loop. Pricing is not in that batch; it comes from an earlier pricing scan; within that batch, turning pricing on raises a single evaluation from 15.05 ms to 72.9 ms, another factor of about 4.8. The absolute timings of the two batches are not directly comparable, but the product of the two multiples shows that the most and least expensive configurations differ by about two orders of magnitude in the cost of a single evaluation. Equal generation counts are therefore not an appropriate comparison, and Section 5 aligns wall-clock time instead.

Under a fixed budget, a more expensive evaluation completes fewer evaluations. Whether an interface converts into a makespan gain cannot be deduced from the design in advance; it must be measured under an equal wall-clock budget. Reservation-table-aware dispatching shows a net gain after the two speed-ups of Section 4.5; the surcharge and yield-record interfaces show no net gain under an equal wall-clock budget.

The framework comes with no convergence proof. Proposition 2, Lemma 1, and Proposition 3 guarantee that every candidate is feasible, that its evaluation is consistent with the conflict model of Section 3, and that the two speed-ups do not change any selection; with the deterministic decode of Section 4.3, the procedure is reproducible. Solution quality is evaluated experimentally in the next section.

5 Experiments

This section answers four questions. How large is the gain from including yielding wait in the fitness (the closed loop)? How much further does reservation-table-aware dispatching reduce makespan, can it replace the closed loop, and do

the two interact positively or combine near-additively? Over what range of instances do these gains hold, and *can the present number of seeds determine* that range? Why do the other interfaces that we implemented fail to yield a net gain?

5.1 Experimental setup

5.1.1 Controlled instance families. FJSP_PT benchmarks usually write travel times as a matrix indexed by location pairs, recording only the total duration between endpoints and supplying no corridor network. Of the 7 layouts used by Homayouni and Fontes [7], 7 matrices are entirely asymmetric (consistent with one-way tracks), and 2 of those also violate the triangle inequality. Asymmetry rules out undirected corridors of the kind used here (one τ_e per corridor, from which the resulting t^* must be symmetric); violation of the triangle inequality is stronger still: the all-pairs shortest paths of any graph satisfy the triangle inequality, so these 2 matrices cannot be recovered even if directed graphs are allowed. Reconstructing a network from such a matrix would produce a different instance, and the published reference values would no longer be comparable. This paper therefore generates a set of controlled instances: the road network is given explicitly, and the features of Section 3.7 can be varied one at a time. A further comparison on public instances—removing the contention constraint and returning to the same mathematical problem as the literature—is in Section 5.9; it calibrates the strength of the upper-level search, not the framework of this paper.

All instances share the same core: 12 jobs, 3 operations per job for 36 real operations, 8 robotic arms, heterogeneity $H = 0.3$, and the transport-to-processing ratio $\bar{T}_t/\bar{T}_p$ scaled to 4.0, so that vehicle contention is the main constraint. H is 0.3 on all ten instances and does not vary with flexibility. The generator draws independent perturbations of the $|\Omega_{ji}|$ processing times of each operation, standardizes them to zero mean and unit variance, and multiplies by H; the within-row coefficient of variation is therefore independent of $|\Omega_{ji}|$, and changing F in Family C does not change H. Small deviations from rounding are measured after generation and recorded with each instance. On this core the instances vary along three axes (Table 4). Instances are constructed by `build()` of `tools/abc_matrix.py`, with a generation seed fixed at 42 and shared by the ten instances. The generation seed is not the run seed: the latter is the set used for the 10 replicates of each instance in Section 5.1.2, namely 42, 7, 2024, 13, 1, 99, 123, 777, 31415, 8—the first coincides in value with the generation seed and differs in role.

Family A: layout (contention structure). A dumbbell is a road network whose two ends are joined by a bottleneck corridor: one end is the load/unload point v_0, the other is the arm area, and only one or a few parallel lanes remain in between. Industrially it corresponds to a spine layout—processing areas line both sides of a central corridor, and in-plant moves travel that spine; on high-volume spine plants, congestion of the automated transport system on the central corridor is a known shop-floor problem [18]. A tandem single-vehicle loop instead splits the guidepath into non-overlapping zones; vehicles within a zone do not meet, and both intersections and head-on conflicts are few [17]. This paper takes the dumbbell because it concentrates contention on parallel lanes that can be added or removed; adding a lane changes capacity only, not distance.

A dumbbell covers only the high-contention range. Covering the low-contention range requires another topology, a *grid*: with more intersections and detours, an occupied corridor can be bypassed and contention falls, but the distance from the load/unload point to the farthest arm grows. The five levels therefore split into two topologies: `funnel`, `high`, and `mid` are dumbbells; `low` and `scatter` are grids. Contention intensity runs from 31.9% to 6.6%, but is not monotone along this construction order (`low` is above `mid`; per-instance values are in Table 4). The paper-wide maximum contention 33.5% is not in this family; it is the instance that takes flexibility to $F = 1.0$ (every operation may use every arm) on the same `funnel` layout, and belongs to Family C below.

The three dumbbell levels only add parallel lanes (load/unload exit 1 → 2, mid-segment 1 → 2): each added lane is a two-hop path of equal total duration, so adding or removing one changes only the number of vehicles that can pass in parallel, not the distance; the node/corridor counts rise from 13/12 to 15/16. `funnel` and `high` differ only by one parallel exit at the load/unload point, which halves the min-cut, and form a single-factor contrast. The two grid levels switch to a 4×4 grid (16/24) and, on the same grid, a scattered placement of arms at the farthest points; the two share size and edge weights and differ only in arm placement.

The unimpeded trip from the load/unload point to the farthest arm is 10.0 on all three dumbbell levels and 18.0 on both grid levels, nearly twice as long. The two grid levels also have the highest makespans in the family, even though their contention intensity is the lowest. This family is therefore a splice of "capacity + topology", not an end-to-end single-factor scan: the difference among the three dumbbell levels can be attributed to capacity; crossing to the grid changes capacity and distance together. The composite lower bound of Table 4 should not be used to judge distance: it takes the maximum of three relaxations, is dominated by the load/unload-cut bound on `funnel` and the two grid levels, and by the job-chain bound on `high`/`mid`, so its jumps need not come from distance. When Section 5.5 orders instances by contention intensity, two inversions appear: one stems from this topological splice, and the other lies inside the dumbbell and needs a separate explanation. Contention intensity is the quantity with which this paper directly describes congestion, and this family has the widest span, so that section's judgement of the closed-loop range of applicability rests mainly on this family.

Family B: vehicle-to-arm ratio N_A/N_M. On `funnel` take $N_A \in \{4, 8, 12, 16\}$, i.e. ratios 0.5, 1.0, 1.5, 2.0, of which 1.5 is the base point shared by the three families (listed as `A funnel` under Family A). This family is used in Section 5.7: the trial-query cost of dispatching grows approximately linearly with the fleet, the pruning effect weakens with it, and the crossing of the two may correspond to a size at which the gain vanishes.

Family C: flexibility F. On the same layout take $F \in \{0.3,\ 0.6,\ 1.0\}$, i.e. the fraction of arms each operation may use, of which 0.6 is likewise the base point. $|\Omega_{ji}|$ is randomized and rounded so that its expectation equals $F \cdot N_M$; the measured base point is 0.61 (Table 4 lists measured values; the other two levels match the targets). Flexibility decides the scope of reassignment, and also how far corridor contention can be turned into an adjustment of the assignment.

The three families comprise 10 instances: the base point `A funnel` is at once 1.5 of Family B and 0.6 of Family C; it is generated and run only once, and occupies only one row in Tables 4 and 7. Hence $5 + 3 + 2 = 10$, while the Family B and Family C scans have four and three levels respectively—Section 5.5 must count the base point when it reads those two families; omitting it would make them look more monotone than they are.

Contention intensity is determined by instance structure and measured on the initial random population (20 individuals, seed 42) as the mean share of yielding wait in travel time; per-instance values are in Table 4. Optimized solutions are not used, because they reflect the solution rather than the instance: the search routes around congested corridors, contention tends to zero after convergence, and the differences among instances disappear. Section 5.5 uses it to order instances.

Comparison with public instances. The H, F, and $\bar{T}_t/\bar{T}_p$ of the self-generated instances need an external reference. The features of the 20 public instances of Section 5.9 are computed with the same feature function and convention, so they can be compared directly. Median flexibility is 0.38; the base point 0.6 of this paper is about one and a half times that, and Family C's $F = 0.3$ falls inside the public range; median heterogeneity is only 0.135, the largest among the 20 instances is 0.258, and none reaches the $H = 0.3$ used here; the public maximum of $\bar{T}_t/\bar{T}_p$ is only 0.12, while this paper scales to 4.0.

Table 4. Controlled instances. All instances share 12 jobs, 36 real operations, 8 robotic arms, $H = 0.3$, and $\bar{T}_t/\bar{T}_p \approx 4.0$; the three families vary only layout, the vehicle-to-arm ratio, and flexibility, respectively. Contention intensity is the share of yielding wait in total travel time on the initial random population; it is determined by the instance structure, not supplied as an input.

Instance	Varied factor	Nodes	Corridors	N_A	F	Contention	Composite LB (time units)
A funnel	layout	13	12	12	0.61	31.9%	24.0
A high	layout	14	14	12	0.61	30.9%	17.0
A low	layout	16	24	12	0.61	11.8%	36.0
A mid	layout	15	16	12	0.61	10.8%	17.0
A scatter	layout	16	24	12	0.61	6.6%	36.0
B NA/NM 0.5	N_A/N_M	13	12	4	0.61	15.8%	24.0
B NA/NM 1.0	N_A/N_M	13	12	8	0.61	30.2%	24.0
B NA/NM 2.0	N_A/N_M	13	12	16	0.61	32.4%	24.0
C F=0.3	flexibility	13	12	12	0.30	27.7%	24.0
C F=1.0	flexibility	13	12	12	1.00	33.5%	24.0

The last two items are both extrapolations, with different meanings. $\bar{T}_t/\bar{T}_p = 4.0$ is deliberate: if transport is only a few percent of processing time, vehicle contention need not be modelled separately, and the common FJSP_PT instances are of this kind. $H = 0.3$ lies above the literature range; if H raised the closed-loop gain, part of the difference among levels would stem from this value. Section 5.5 tests this with independent data spanning four H levels and finds that the gain does not rise with H; the instances and baseline of that batch differ from the main batch, so it supports only this directional conclusion.

5.1.2 Comparison protocol. The comparison rests on the following rules.

Common executor: report the realized makespan. The makespan that the open loop produces from unimpeded travel times is often infeasible on the real corridors. Every configuration's schedule is therefore executed by the same conflict-free path planner and then checked by a validator independent of the decoder; what is compared is always the realized makespan after execution. The gap between the open loop's self-reported and realized values (hereafter, the optimism margin) is especially large on the funnel layout. If each configuration were compared on its self-reported value, the open loop would gain an advantage from a makespan that cannot be realized.

Equal wall-clock budget, not equal generation count. What a single evaluation does differs by configuration: the open loop looks up the constant matrix t^*; the closed loop runs conflict-free routing for every candidate; pricing further requires a multi-label search. The cost of a single evaluation can differ by about two orders of magnitude. Comparing at equal generation count would give the more expensive configuration more compute, and an apparent mechanism gain might be only a larger budget. This paper aligns wall-clock time: the same instance shares the same budget; the main batch is 90 seconds, with a generation cap of 2000 and early stopping after 400 consecutive generations without improvement. The generation cap never fires; every open-loop run is terminated by early stopping after a little over four hundred generations, using on average about one fifth of the budget; both closed-loop configurations use the full 90 seconds every time. This setting favours the baseline: the open loop is not cut off by the budget but searches until it no longer improves, and in a shorter wall-clock time it completes as many evaluations as closed-loop rule-based dispatching, or more (Section 5.7). The reason for stopping is recorded run by run in a cost-diagnosis batch with the same budget and hyperparameters; the main batch keeps only the total time per instance, which is consistent with that batch's distribution. The remaining results come from four further batches—convergence curves, a coordination-strength scan,

an independent matrix of 16 instances, and the public benchmarks—each with its own budget and seed set, stated at first citation.

Aligning wall-clock time lets the configuration with the cheaper evaluation complete more evaluations. Every configuration therefore reports both the cost of a single evaluation and the number of evaluations actually completed, and Section 5.7 analyses the trade-off between the two.

Seeds and paired tests. A single seed cannot support a conclusion: on the same instance and configuration, a range of 11 makespan units was observed among three seeds, larger than any mechanism effect being measured. Each instance is therefore run with 10 seeds, for $10 \times 10 = 100$ pairs in the aggregate, and a two-sided Wilcoxon signed-rank test is performed over seeds. Pairs are formed by the key (instance, seed) rather than by list position, so that pairs are not misaligned when some instance is incomplete. 10 seeds are enough to resolve an effect of a few percentage points on the 100 pairs in the aggregate, but not enough to resolve it inside a single instance. Section 5.5 shows that this boundary falls exactly between the closed loop and dispatching; per-instance conclusions are confined to this power.

Multiple comparisons and effect sizes. The paper contains a little over twenty Wilcoxon tests. Without correction, "at least one significant among ten instances" is about 40% under the global null, not the nominal 5%. Family-wise tests are always corrected by family; the family is stated where the result is reported. There are two kinds of family: the five aggregate contrasts of Table 6 form one family; in Table 7, the 10 instances of one effect form a family each. Correction uses Holm, to control at least one Type I error inside the family; Benjamini–Hochberg is computed as well, and is marked where the two conclusions differ.

Two exceptions. First, Section 5.5 reports the vehicle-to-arm ratio with uncorrected p-values and marks each one: these values are already non-significant before correction, and correction would only make the conclusion more conservative. Second, the independent data of 16 instances cited in Sections 5.5 and 5.6 take their count of significant instances from that batch's own report, not corrected by the rules of this paper; they serve only as a check of direction and magnitude, and every mention of that count is to be read as an uncorrected count. Apart from these two places, every p-value in this paper is corrected.

Correction only answers whether the direction can be distinguished from zero. Each aggregate contrast additionally reports a Hodges–Lehmann pseudomedian and a 95% distribution-free confidence interval: it is of the same source as the Wilcoxon signed-rank test; the point estimate, the interval, and the p-value are read from the same null distribution. The mean pairwise relative gain used as the main column of the tables is convenient for adding across tables, but it is not of the same source as the rank test and carries no precision. Both are given: for example the mean of dispatching inside the closed loop is 2.45% and the pseudomedian is 3.03%; the gap comes from pairs on two vehicle-to-arm-ratio instances where makespan worsens (positive gain), to which the pseudomedian is insensitive. Error bars in the figures are the standard error of the mean of pairwise relative gains; shaded bands are the seed-to-seed range; tables do not report ±.

Correctness. Each solution has its corridor occupancies reconstructed from the timetable by a validator independent of the decoder, and is compared with the composite lower bound. Runs that fail the check or fall below the bound are reported separately and are not folded into any instance mean. Completed runs are appended to a persistent ledger; an interrupted batch can be resumed from a checkpoint, and reports are rebuilt from the ledger.

5.1.3 Implementation and environment. The experiments run single-threaded on an Intel Xeon W-1270 @ 3.40 GHz; the implementation is pure Python (≥ 3.9.6), uses only the standard library, has no third-party dependency, and does

Table 5. The 2×2 structure of the four-level ladder. Rows are whether the search counts conflict-induced delay; columns are whether a vehicle is selected by unimpeded time or by a trial query on the reservation table. Comparisons within a row or a column differ in one factor; a diagonal comparison changes both and cannot be decomposed.

	Rule-based dispatching	Reservation-table-aware dispatching
Open-loop planning (search ignores conflicts)	B0 [10]	B0+ [19]
Closed-loop planning (fitness from conflict-free routing)	B1	**B2** (this paper)

not depend on a solver. Hyperparameters of the outer genetic algorithm and the local search take their defaults, are not tuned, and are identical across configurations: population 60, crossover probability 0.8, mutation probability 0.2, elite 5, binary tournament; local search acts each generation on the top 10% of the population, examines at most $L = 5$ critical operations per round, and at most three rounds. The hyperparameters are conventional and identical across configurations, so measured gains come from the mechanisms rather than from operator tuning. Absolute runtimes cannot be compared with a compiled implementation: a faster implementation would let every configuration complete more evaluations within the same budget, but would not change the ratio of cost per evaluation among configurations, which is measured among configurations sharing the same implementation.

5.2 Four-level baseline ladder

The comparison must answer two questions separately: how much makespan differs when the structure is aligned with a published method, and how much it differs when only one component is changed. The former requires structural correspondence; the latter requires changing one component at a time. Ablations alone cannot be mapped onto published structures, while a literature comparison alone would credit the gain of conflict-free routing itself to our mechanisms. We therefore use a four-level ladder in which each level corresponds to an identifiable structure.

B0: open-loop planning + realized execution. The upper level searches by unimpeded shortest paths (the search ignores conflicts) and, after the search, is executed by the same conflict-free path planner. This corresponds to [10]: the scheduling layer produces a Gantt chart that ignores transport contention; a downstream priority router resolves conflicts and then shifts arrival times back onto the time axis; dispatching is idle-first $k^* = \arg\min \mathrm{Cost}(\mathrm{path}_k, L_{\mathrm{start}})$, the same formula as the rule-based dispatching (12) of this framework.

B0+: open-loop planning + reservation-table-aware dispatching. The schedule is likewise fixed by unimpeded travel times; only vehicle selection is changed to a trial query on the reservation table. This corresponds to the two-stage structure of [19]: a hybrid genetic algorithm in the first stage schedules on a *constant travel-time matrix*, and only in the second stage, on a frozen schedule, are vehicle assignments optimized and conflicts handled, with no feedback. Reservation-table-aware dispatching is used here as a greedy approximation of that second stage.

B1: closed-loop planning + rule-based dispatching. Fitness during the search is taken from conflict-free routing; dispatching still uses the unimpeded matrix of (12). The only difference from B2 is the vehicle-selection rule.

B2: closed-loop planning + reservation-table-aware dispatching. The method of this paper.

The four levels form a 2×2 design along two factors rather than a linear sequence: whether the search counts conflict-induced delay (open-loop / closed-loop), and whether a vehicle is selected by unimpeded time or by a trial query on the reservation table (rule / trial query). If two configurations differ in both factors, their difference cannot be decomposed. For example, B0+ $\longrightarrow$ B1 crosses both factors and **cannot** be used to measure the closed loop. The interpretable contrasts fix one factor and change only the other, i.e. the four pairs sharing a row or a column of Table 5.

Table 6. Makespans of the four-level ladder under an equal wall-clock budget, and the four pairs that change only one place along the two factors. A negative value means a lower makespan, i.e. better. $n = 100$ comparisons paired by (instance, seed), two-sided Wilcoxon signed-rank tests; p-values have received a Holm correction over the five contrasts of this table ($^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$). The HL column is the Hodges–Lehmann pseudomedian of pairwise relative gains and its 95% distribution-free confidence interval (see Section 5.1.2). A bold header marks our configuration; numbers are not bold because B2 is not best on every instance (per-instance results in Table 7).

		$C_{\max}$ (time units)			
Instance	Contention	B0	B0^{+}	B1	**B2**
A funnel	31.9%	58.10	53.50	44.70	42.40
A high	30.9%	56.00	51.40	39.90	37.70
A low	11.8%	64.50	64.60	57.80	57.10
A mid	10.8%	46.40	42.70	39.40	37.40
A scatter	6.6%	62.30	62.30	57.20	57.10
B NA/NM 0.5	15.8%	71.00	70.50	69.00	65.10
B NA/NM 1.0	30.2%	56.70	56.50	47.50	49.80
B NA/NM 2.0	32.4%	70.60	64.20	42.20	44.20
C F=0.3	27.7%	74.30	69.50	51.60	47.30
C F=1.0	33.5%	52.10	47.50	39.30	36.30
Pooled mean	—	61.20	58.27	48.86	47.44

Single-factor paired contrast	$\Delta C_{\max}$	HL pseudomedian and 95% CI (%)	W/L/T	p_{Holm}
Closed-loop value (rule-based)	$-18.86\%^{***}$	-18.28 [-21.95, -14.68]	96/2/2	$< 10^{-4}$
Closed-loop value (reservation-table-aware)	$-17.75\%^{***}$	-18.00 [-20.55, -15.22]	93/5/2	$< 10^{-4}$
Dispatching value (open-loop)	$-4.59\%^{***}$	-4.45 [-6.13, -2.88]	68/19/13	$< 10^{-4}$
Dispatching value (closed-loop)	$-2.45\%^{**}$	-3.03 [-5.19, -0.94]	60/34/6	0.0054
End-to-end: literature → this paper	$-21.35\%^{***}$	-21.20 [-24.46, -17.99]	94/5/1	$< 10^{-4}$

The execution convention of B0 decides how strong the baseline is. The plan produced by the open-loop search already contains a set of vehicle assignments; execution has two reproductions: *replay the plan* executes the vehicles specified by the original plan; *on-site reassignment* re-selects vehicles by the rule at execution time. The former is measured to be stronger (63.00 versus 78.00 on the funnel instance) and corresponds to [10]: that paper's task-allocation layer fixes the robots on the Gantt chart first, and execution only shifts the time axis, without reassignment. The main report takes the stronger replay-the-plan convention; on-site reassignment is listed alongside.

5.3 Aggregate comparison on the ladder

Table 6 gives the makespans of the four-level ladder on 10 instances × 10 seeds, for 100 pairs in all, each instance under the same wall-clock budget (Section 5.1.2); every run passed the independent check. The next three paragraphs read the two factors separately, and then the aggregate B0 → B2. The p-values in the table are Holm-corrected over these five contrasts, and the correction changes no conclusion. The text reports the pseudomedian and confidence interval as a *reduction*; Table 6 gives the same interval as negative values.

Closed-loop evaluation. Fixing the vehicle-selection rule and changing only whether the search includes conflict-induced delay, the closed loop is significantly better under both dispatching rules: under rule-based dispatching B0 → B1

is -18.86% ($p < 10^{-4}$), and under reservation-table-aware dispatching B0+ $\rightarrow$ B2 is -17.75% ($p < 10^{-4}$). The Hodges–Lehmann pseudomedian of the former is 18.28%, with 95% confidence interval [14.68%, 21.95%]: the interval lies far from zero as a whole, and its lower end is still a two-digit reduction, so this level does not depend on whether the point estimate is a mean or a pseudomedian. The gap cannot be explained by the number of evaluations, since that difference runs the other way: an open-loop evaluation only looks up a constant matrix, and inside the same budget it completes several times as many evaluations as the closed loop (cost per evaluation of each configuration is in Section 4.5). The open loop evaluates more and is still worse, because it optimizes a surrogate that excludes yielding wait; the true transport cost appears only after the plan is fixed. Figure 3 draws this gap on a single instance.

Closed-loop evaluation is also the follow-up direction listed by [10] in its conclusions ("extending this mechanism into a fully iterative loop"). The closed loop is significant under both dispatching rules, so this conclusion does not depend on our vehicle-selection rule.

Reservation-table-aware dispatching. Fix the open or closed loop and change only the vehicle-selection rule. Inside the *closed loop*, B1 $\rightarrow$ B2 is -2.45% ($p = 0.0054$), with pseudomedian 3.03% and 95% confidence interval [0.94%, 5.19%]. The two levels share the same scheduling search and the same budget; they differ only in whether vehicle selection uses a trial query on the reservation table. This is a pooled result: the interval does not cross zero, and its lower end is less than one percentage point from zero; after family-wise correction no instance is significant (2 before correction), and two of the ten instances have a positive sign, i.e. B2 is slower than B1 at vehicle-to-arm ratios 1.0 and 2.0 (Table 7). Thus -2.45% is an average effect over this instance family and cannot be applied instance by instance; Section 5.5 discusses this resolution limit.

Under the *open loop*, the same rule is -4.59% ($p < 10^{-4}$), likewise significant. It corresponds to the segment of [19]'s two-stage structure relative to the open loop of [10]; the magnitude (4.59%) is far smaller than that of the closed loop (18.86%). Reservation-table-aware dispatching is significant under both the open and the closed loop, but much smaller, and cannot by itself explain the B0 $\rightarrow$ B2 reduction.

End-to-end. B0 $\rightarrow$ B2 is -21.35% in the aggregate ($p < 10^{-4}$; of the 100 pairs, 94 wins, 5 losses, 1 tie). That is, moving from open-loop planning with rule-based dispatching to closed-loop planning with reservation-table-aware dispatching reduces makespan by about one fifth; the two paragraphs above give its components.

5.4 Near-additivity and non-substitutability

The three effects of the previous section differ in magnitude: changing only vehicle selection under the open loop gives 4.59%, significant but small; changing only whether the search counts yielding wait, under rule-based dispatching, gives 18.86%, significant and large; changing vehicle selection again inside the closed loop gives a pooled mean of 2.45%, reduced by the two vehicle-to-arm-ratio instances 1.0 and 2.0 on which B2 is slower than B1. This section answers two questions separately: whether the two can substitute for each other, and whether they interact positively.

Interaction. If the closed loop and dispatching were independent of each other, the end-to-end gain would equal the compound of the two separate gains: from the closed loop under rule-based dispatching (18.86%) and dispatching under the open loop (4.59%) one obtains 22.6%. The measurement is 21.35%, slightly below that prediction—*no positive interaction is detected*, and the two are near-additive. They do not amplify each other.

Non-substitutability. Changing only vehicle selection under the open loop cannot replace the closed loop. Fixing the schedule by unimpeded travel times and selecting vehicles only from the reservation table yields about 4.59%; including

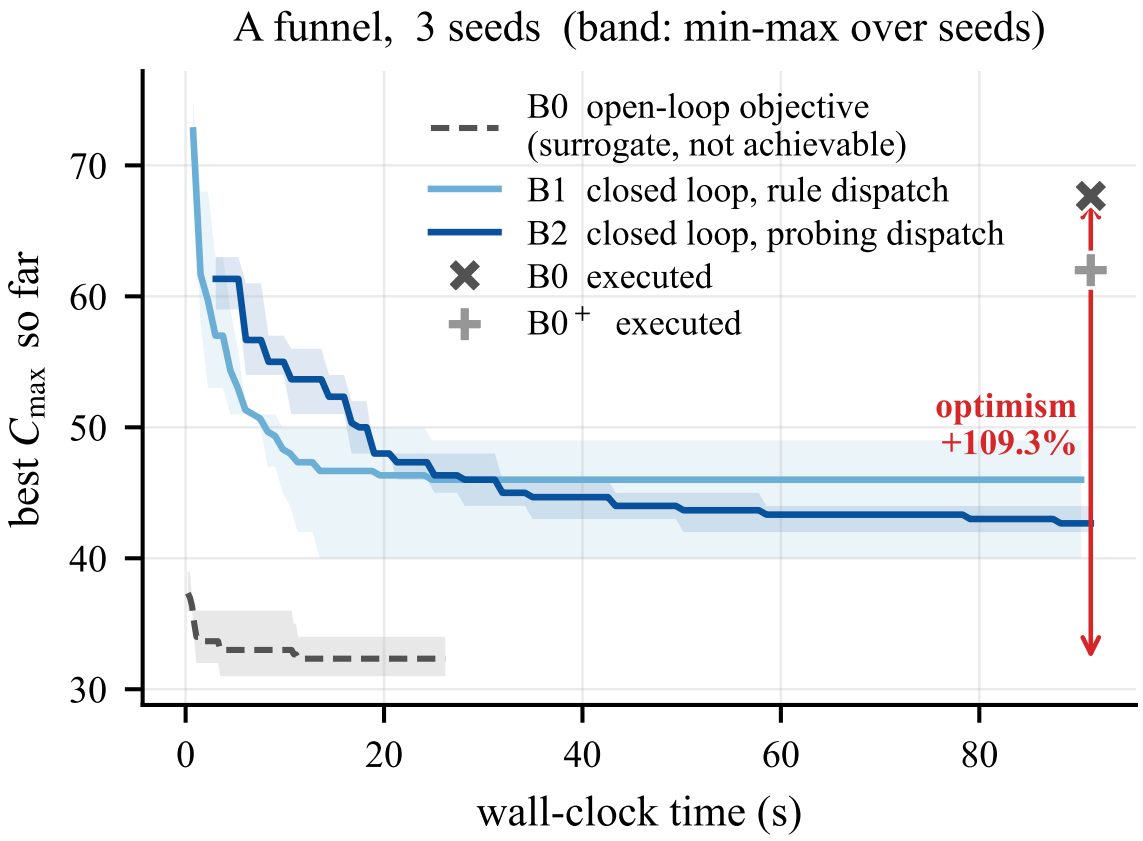


Fig. 3. Convergence curves on a single `funnel` instance (taken from a diagnostic batch of three instances); the horizontal axis is wall-clock time, the curves are the time-wise mean over three seeds, and the shaded band is the seed-to-seed *range* (the sample is too small to estimate quartiles). The solid lines (B1, B2) are always realized makespans; the dashed line is the open loop's surrogate objective, i.e. the quantity it actually optimizes; crosses and pluses mark the makespan of the same plan after conflict-free routing. The gap between the end of the dashed line and the cross is the optimism margin of that instance: the open loop does not converge poorly; it converges to a surrogate that cannot be realized. The figure is illustrative and supports no statistical conclusion (statistics use the main batch of 10 instances × 10 seeds); its wall-clock budget is 46.3 seconds, different from the 90 seconds of the main batch.

yielding wait in the search and still using rule-based dispatching yields about 18.86%. Placed in a two-stage structure of the [19] kind, the same dispatching rule yields a gain only of the open-loop dispatching magnitude; to obtain the closed-loop reduction, yielding wait must enter the fitness during the search. B2 is best end to end because the two gains add up, not because of a positive interaction.

Relation to schedule-then-resolve. The shortfall of this route lies not in second-stage dispatching (which yields a significant 4.59% under our protocol) but in the first stage not counting yielding wait, so the gain from closed-loop evaluation is unavailable. This difference is measured on the instance family of this paper.

5.5 Three-factor scans and per-instance resolution

The aggregate comparison of Section 5.3 rests on 10 instances. This section takes them apart by layout, vehicle-to-arm ratio, and flexibility. The trend of the closed loop can be interpreted by family, whereas dispatching cannot be distinguished from zero on a single instance; this section reports that difference. Each of the three families corresponds to an expectation that can be stated in advance.

Layout (Family A) tests whether the gain grows with contention intensity: both paths target contention, and their effect is limited when contention is weak. `funnel` and `high` differ only by one parallel exit at the load/unload point, which halves the min-cut, and form a single-factor contrast.

Vehicle-to-arm ratio (Family B) tests whether there is an upper bound on fleet size beyond which overhead offsets the gain. Trial-query cost grows approximately linearly with the fleet; the pruning of Section 4.5 has its lowest cut rate on the largest fleet. The four levels are $N_A/N_M \in \{0.5, 1.0, 1.5, 2.0\}$, of which 1.5 is the shared base point `A funnel`.

Flexibility (Family C) tests whether the gain depends on room to reassign. $F \in \{0.3, 0.6, 1.0\}$, of which 0.6 is likewise the base point.

Per-instance resolution. The significance marks of Table 7 decide which instances may be read by sign. Testing each of the 10 instances once, "at least one significant" under the global null is far above 5%. Per-instance tests therefore receive a Holm correction by family, the family being the 10 instances of one effect (the closed loop and dispatching each form a family; they answer different questions and are not merged into one family of twenty tests). The marks of Table 7 and Figure 4 are all based on the corrected p.

After correction the closed loop remains significant on 9 instances (9 before correction; the largest corrected value is $p = 0.0401$, so the conclusion does not depend on the correction); the only exception is the instance with the fewest vehicles (B0 $\rightarrow$ B1 is -2.0%, not significant). The trend of the closed loop can therefore be interpreted by family. Dispatching inside the closed loop is significant on 2 instances before correction (the two ends of Family C, $p = 0.0105$ and $p = 0.0321$), and *on 0 instances after correction*: Holm gives $p = 0.1052$ and $p = 0.2885$, Benjamini–Hochberg $p = 0.1052$ and $p = 0.1603$, likewise not significant. No instance is significant on its own. Its magnitude (a few percentage points) is of the same order as the smallest effect that 10 seeds on one instance can resolve; only the 100 pooled pairs distinguish it from noise, and only just: the pseudomedian is 3.03%, the 95% interval $[0.94\%,\ 5.19\%]$ does not cross zero, and its lower end is less than one percentage point from zero. Below, per-instance results for dispatching are reported only as directions; the sign is judged only on the pooled data.

Table 7 and Figure 4 give the per-instance results. The three families are read differently.

Layout family. The closed loop rises roughly with contention intensity and is significant on all five instances: on the least congested `scatter` (contention 6.6%) B0 $\rightarrow$ B1 is -8.1%; on `mid`/`low` it is -14.4%/-10.3%; on `funnel`/`high` it is -21.3%/-26.6%. "Roughly" because there are two inversions of different kinds. `low` has higher contention than `mid` but a smaller gain; the two lie across the topological boundary of Section 5.1 (grid versus dumbbell, farthest trip 18.0 versus 10.0), so this is not a pure contention inversion. `funnel` has higher contention than `high` but a smaller gain: both are dumbbells differing only by one parallel exit at the load/unload point, the planned single-factor contrast, so *this inversion is a counter-example*. What can be established is the magnitude (the most congested instance -21.3% versus the least congested -8.1%, about two and a half times), not a monotone relation between contention and gain. Contention intensity summarizes congestion as a single number and does not separate the part that reassignment can bypass; see limitation (1) of Section 6.2. Dispatching is not significant on any instance of this family; only its direction is reported: four of the five instances are negative, the same direction as the closed loop.

Table 7. Per-instance four-level ladder results and the two effects of the closed loop and of dispatching, blocked by the three families (each instance is $n = 10$ comparisons paired by seed). Significance marks are given by the **Holm-corrected** p (two-sided Wilcoxon signed-rank test); the family is the 10 instances of one effect: $^{*}p < 0.05$, $^{**}p < 0.01$; † means significant before correction and not significant after, and the sign of that instance is not taken as a conclusion. A negative value means a lower makespan. The two gain columns use the same estimator as Table 6 (the mean of pairwise relative gains over seeds) and cannot be obtained by dividing the two means in the same row. `A funnel` is at once $N_A/N_M = 1.5$ of Family B and $F = 0.6$ of Family C; it is run only once and occupies only one row.

		$C_{\max}$ (time units)				Pairwise relative gain	
Instance	Contention	B0	B0^{+}	B1	**B2**	B0→B1	B1→B2
A funnel	31.9%	58.10	53.50	44.70	42.40	−21.3%*	−4.6%
A high	30.9%	56.00	51.40	39.90	37.70	−26.6%*	−4.8%
A low	11.8%	64.50	64.60	57.80	57.10	−10.3%*	−1.1%
A mid	10.8%	46.40	42.70	39.40	37.40	−14.4%*	−4.8%
A scatter	6.6%	62.30	62.30	57.20	57.10	−8.1%*	+0.1%
B NA/NM 0.5	15.8%	71.00	70.50	69.00	65.10	−2.0%	−4.7%
B NA/NM 1.0	30.2%	56.70	56.50	47.50	49.80	−15.6%*	+5.0%
B NA/NM 2.0	32.4%	70.60	64.20	42.20	44.20	−37.7%*	+5.7%
C F=0.3	27.7%	74.30	69.50	51.60	47.30	−29.7%*	−8.2%†
C F=1.0	33.5%	52.10	47.50	39.30	36.30	−22.9%*	−7.2%†

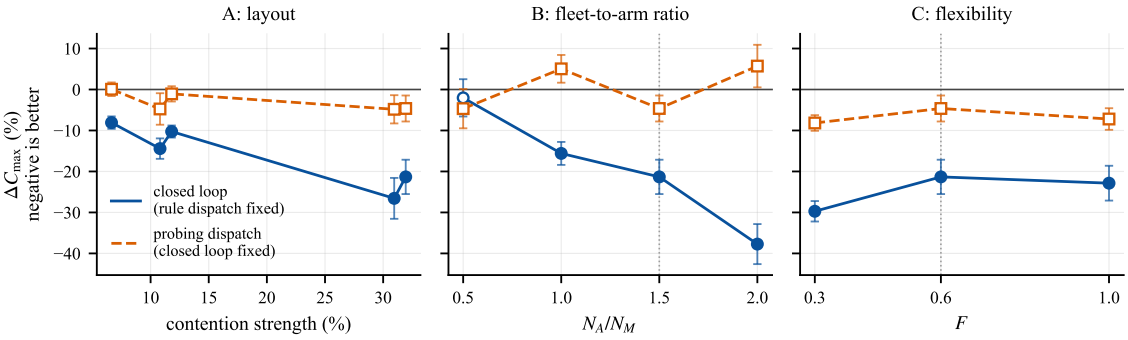


Fig. 4. Distribution of the closed loop and of dispatching on the three families. Left: Family A (layout), horizontal axis contention intensity; middle: vehicle-to-arm ratio N_A/N_M; right: flexibility. Families B and C include the shared base point (1.5 and 0.6 at the hollow vertical lines), also listed as `A funnel` in Family A. A filled marker remains significant after Holm correction ($p < 0.05$, the family being the 10 instances of one effect); a hollow marker is not significant. Error bars are the standard error of the mean of pairwise relative gains over the 10 seeds of that instance. The closed loop is significant on 9 instances and rises roughly with contention; after correction dispatching is significant on *no instance*, the error bars cross zero on every instance, and the sign alternates across the four vehicle-to-arm-ratio levels. The point of the figure is whether the two curves can be distinguished from zero, not the shape of the dispatching curve.

Vehicle-to-arm-ratio family. This family does not determine an upper bound of applicability for dispatching. Dispatching inside the closed loop at the four levels is, in order, −4.7% ($N_A/N_M = 0.5$), +5.0% (1.0), -4.6% (1.5, the base point `A funnel`), +5.7% (2.0): the sign alternates, and none of the four instances is significant ($p = 0.4121$, $p = 0.2011$, $p = 0.1386$, $p = 0.3313$; these are values *before* correction and are already non-significant). Taking only the three levels 0.5, 1.0, and 2.0, the curve shows the monotone rise predicted by the cost law; with the middle base point included, this

trend no longer holds. When the effect is of the same order as per-instance noise, *subset selection alone can produce a spurious trend*; the safeguard is a complete scan with per-instance tests. The upper bound predicted in Section 5.7 therefore remains untested: testing it would require enough seeds to resolve effects of a few percentage points, beyond the budget of this paper. The closed loop at 1.0 and 2.0 is -15.6% and -37.7%, both significant: fleet size mainly affects the closed-loop gain, not dispatching.

Flexibility family. After Holm correction, the dispatching effect is not significant in this family either. At $F = 0.3$, 0.6, and 1.0, dispatching is -8.2%, -4.6%, and -7.2%: the three have the same sign, and the two ends are the two largest magnitudes among all 10 instances (the base point falls back to the level of the rest); the two ends $p = 0.0105$, $p = 0.0321$ rise after correction to $p = 0.1052$, $p = 0.2885$ and are no longer significant. This family supplies only directional evidence of the same sign as the aggregate and of the largest magnitude. The same sign at all three levels, without the sign alternation of the vehicle-to-arm-ratio family, is compatible with "dispatching acts mainly through vehicle selection and timing and does not depend strongly on the number of eligible arms": if dispatching relied on room for reassignment, $F = 0.3$ should fail first, yet it has the largest magnitude. The closed loop is -29.7% and -22.9%, still significant per instance after correction.

An independent batch of self-generated instances. The results above come from one instance family and one baseline. A cross-check uses another batch of self-generated instances with contention (public benchmarks are in Section 5.9, where contention has been removed): 16 instances × 10 seeds, 1280 runs in all. Both the instance core and the baseline differ: the baseline is two-stage surrogate transport, crossed on four contention levels and four heterogeneity levels. The budget and the seeds differ as well: wall-clock time is calibrated per instance from 30.2 to 72.9 seconds (configurations inside the batch share the same calibration, so among configurations it is still an equal budget), and another ten seeds are used. The *integration gain* (two-stage → closed-loop) is 5.52%, significant on 11 of the 16 instances, ranging from 0.28% to 9.28%, consistent in direction and magnitude with the closed loop of this paper. Counts of significant instances in this batch are uncorrected (see Section 5.1.2).

Heterogeneity does not inflate the closed-loop gain. The four H levels of that batch answer the question left by Section 5.1. Splitting the 16 instances into four bins by H (four instances per bin, one from each contention level), the integration gain is: 6.12% at $H = 0$, 5.62% at $H = 0.15$ (measured 0.139), 5.95% at $H = 0.3$, 4.38% at $H = 0.5$; the rank correlation of gain with H is a weakly negative -0.21.

The measured $H = 0.139$ falls near the public-instance median 0.135, and its 5.62% differs from the 5.95% at $H = 0.3$ by less than half a percentage point: setting H above the literature range did not inflate the closed-loop gain. The expectation that the gain rises with H does not hold—the trend is not monotone, and the rank correlation is negative. Each bin has only 4 instances, and the within-bin spread (0.28% to 9.28%) is far larger than the gap among bins, so this supports only "the gain does not rise with H" and is not sufficient to assert that $H = 0.5$ is lower. This is compatible with the analysis of Section 3.7: as H grows, processing-time differences among optional arms gradually outweigh differences in corridor reachability, so assignment is determined increasingly by processing speed and decreasingly by traffic conditions. $H = 0$ closes the trade-off, not the opportunity: processing times inside Ω_{ji} are identical, reassignment need not trade processing time for traversal time, and it becomes a pure congestion decision of zero cost; that bin has the highest gain of the four (6.12%), consistent with the definition. Thus H makes arm selection a trade-off (without heterogeneity the reversal of Section 1 would not occur), but it does not amplify the closed-loop gain. The closed-loop gain comes from counting yielding wait during the search; contention strength is determined by layout and fleet, not by H.

That batch gives reservation-table-aware dispatching -0.74%, significant on 0 of the 16 instances, the opposite direction from 2.45% in this section. The most likely reason is that the batch ran before the two speed-ups of Section 4.5, when the overhead of the rule had not yet been reduced (the controlled comparison in Section 5.7 supports this explanation).

5.6 Ablations and controlled negative controls

Section 4.4.1 listed three choices for the inter-layer interface. The default configuration uses only the first (realized durations); the other two have both been implemented and measured, and neither has a net gain under an equal wall-clock budget. This section reports these results.

5.6.1 Yield-record-guided local search. On the 16-instance data of Section 5.5, turning off yield-record feedback has a relative gain of -1.72% (significant on only 1 of the 16 instances), and turning off the staggering operator has -0.16% (significant on 2 instances). Neither is positive: under an equal wall-clock budget, decodes spent on this neighbourhood are better spent on population evolution. The mechanism is explained in Section 5.6.3.

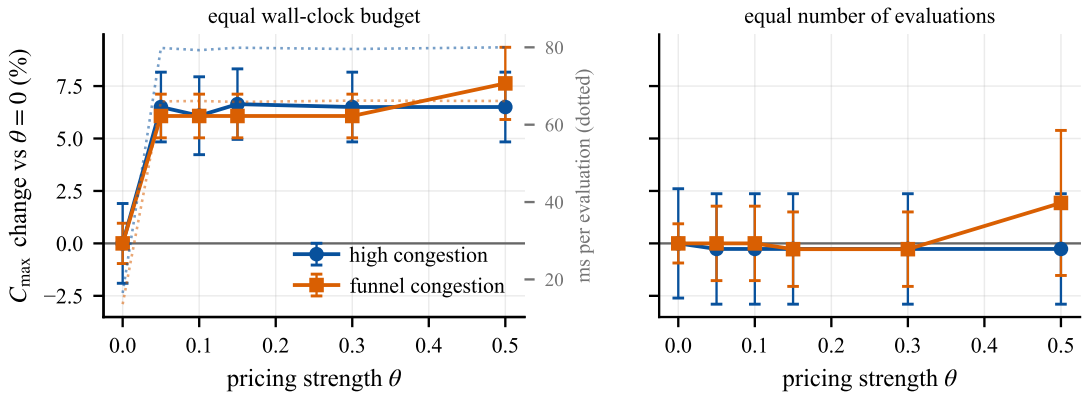
These two ablations may cite that earlier batch; the dispatching ablation may not: the yield record and staggering were not changed before or after the two speed-ups, and their cost does not vary with pruning; the cost of dispatching is precisely what the speed-ups target.

5.6.2 Surcharge on corridor–time slots. Section 4.4.1 noted that attaching a further cost to already occupied corridor–time slots counts again a delay that fitness has already included. That argument shows that a surcharge adds no information, but it cannot give the size of the makespan loss, which must be measured separately. θ is scanned under both budget conventions, and the search width of path planning is held fixed at every θ, so that the surcharge is not mixed with "how many entry times are examined".

The measurements are in Figure 5. Under equal wall-clock time, every θ tried makes the makespan worse and significantly so, and the worsening is flat over a ten-fold range of θ; under an equal number of evaluations, the same comparison cannot be distinguished from zero. Routing with a surcharge completes about one fifth as many evaluations within the same budget as routing without one: the surcharge does not mis-rank candidates, but greatly reduces the number of evaluations within the same budget.

Two further observations rule out other explanations. First, θ is not an effective tuning parameter on this problem: over two orders of magnitude, priced routing modifies the same few paths, pays the same total price, and returns the same makespan; the cumulative price is only a small fraction of total travel time, not enough to change the order of arrival times in arrival time + $\theta \cdot$ price. Only when θ is so large that price outweighs arrival time does double-counting take effect, and then makespan worsens sharply. Second, the overhead is structural rather than caused by a non-zero price: the runtime ratio of the same decode with and without a price is the same at every θ, the extra cost coming from "maintaining a Pareto front at every node and trying several entry times on every arc", regardless of whether a price is actually applied. This agrees with Lemma 1: with a second criterion there is no total-order dominance, and single-label search must be replaced by multi-label search.

The two budget conventions differ on the size of the loss but agree on its sign. Whether the price comes from a finite difference of corridor capacity or from a critical-chain proxy, the delay it targets is already counted in the yielding vehicle's arrival time. As long as fitness is taken from a complete conflict-free decode, the added price carries no new information, and the comparison is decided by the cost of the multi-label search it requires. Hence, before extending the inter-layer interface, one should check whether the receiving layer's objective already contains the quantity the interface would pass; if it does, such an extension is harmful under our protocol.



positive is worse. 20 runs per point on the left, 10 on the right; two instances, mean ± s.e.

Fig. 5. A surcharge on corridor–time slots. A positive value means worse. Under equal wall-clock time (left), the makespan worsening appears as soon as the surcharge is turned on, then stays flat over a ten-fold range of θ, in step with the cost per evaluation on the right axis; under an equal number of evaluations (right) the worsening disappears. The damage comes from a higher cost per evaluation and fewer evaluations inside the same budget, not from the price choosing the wrong path. Markers are means, error bars are the standard error of the mean, and the number of runs per point is marked in the figure footer; "significant" means a two-sided Wilcoxon signed-rank test of each θ against $\theta = 0$, paired by seed. This figure uses two instances of 8 jobs × 4 machines × 4 vehicles, which do *not* belong to Families A/B/C of Table 4 (those are uniformly 12 × 8 × 12); the legend labels `high`/`funnel` are layout tags only, and they are not the same instances as the like-named pair in Family A. The left panel has 10 seeds and the right panel 5 (a subset of the former); the wall-clock budgets are 48.9 and 60.7 seconds, both different from the 90 seconds of the main batch. The batch was collected *before* the two speed-ups; the surcharge mechanism and its cost are unaffected by pruning, so the conclusion still holds, but the results cannot be set beside the other tables of this section instance by instance.

5.6.3 Dilution and overhead. Why the yield record and staggering fail to offset their overhead can be established by measurement; the same measurement also explains why the overhead of reservation-table-aware dispatching had to be reduced first.

Allocation of decodes. Yield-record-guided local search consumes a large share of all decodes, and of the neighbours it decodes, fewer than one in fifty improve makespan. These improvements are real, but the same decodes spent on population evolution yield more.

The neighbourhood is by construction small and well targeted (Section 4.6); a low improvement rate despite good targeting means that the operations named by the record do not shorten the critical chain after reassignment or staggering. We tried three refinements: scoring reassignment by expected yield on the path rather than by unimpeded distance, replacing the adjacent swap of staggering by a directed insertion, and accepting plateau moves that leave makespan unchanged but lower total yield. Each was compared independently with the original operator under an equal number of evaluations, and *none* was significant. The failure therefore lies in the signal itself, not in how the signal is used.

Dilution along the critical chain. A yield record names the corridor, the time slot, and the transport that first occupied that corridor for a given yield, i.e. the information needed to remove *that* event. Makespan is determined by a chain; after one corridor delay is removed, another constraint (an upstream operation, a robotic arm, or another corridor) usually becomes tight, so the realized improvement is only part of the delay removed. This is why the acceptance rule

re-extracts the critical chain after every acceptance. When a bottleneck is only one of several of comparable magnitude, feedback that names it improves makespan by far less than the yield duration removed.

Why dispatching behaves differently before and after the speed-ups. The same dilution acts on reservation-table-aware dispatching: the trial query selects the vehicle with the earliest delivery for the current task, a local improvement of which only part survives transmission through the critical chain to makespan. After dilution the reduction is still positive, but only a few percentage points. Before the speed-ups, a single evaluation costs 12.2 times as much as under rule-based dispatching, too few evaluations fit into the same wall-clock time, and the net effect cannot be distinguished from noise (the independent batch of Section 5.5). After Section 4.5 lowers the cost to 5.23 times, the same reduction becomes significant when pooled.

Under a fixed budget, two quantities that can be measured and improved separately must be considered together: the makespan reduction remaining after dilution, and the extra cost per evaluation. The two mechanisms fail for different reasons. Yield-record-guided local search is limited by the former: too little reduction survives dilution, and the three neighbourhood refinements are not significant under an equal number of evaluations. Dispatching is limited by the latter: the post-dilution reduction is always present, and it was initially not significant because the cost per evaluation was too high, a cost that can be reduced without changing the selected vehicle or the timetable. The default configuration therefore keeps dispatching and does not use the yield record to generate neighbourhoods.

5.7 The cost law and the effect of the speed-ups

Under a fixed wall-clock budget, the cost per evaluation and the number of evaluations trade off against each other. The same dispatching rule has a net effect indistinguishable from noise before the speed-ups and significantly positive after them, so this section measures both quantities directly.

Two budget conventions. Figure 6 gives the cost per evaluation of each configuration and the number of evaluations actually completed within the same budget cap. The cost spans about two orders of magnitude: cheaper configurations complete more evaluations, and the open loop completes the largest number while using only about one fifth of the wall-clock time. Comparing at an equal number of generations gives more compute to the more expensive configuration; aligning wall-clock time lets the cheaper configuration complete more evaluations. Both conventions are therefore reported.

Figure 7 runs the same set of comparisons once under each convention. The equal-generation convention ignores the extra cost per evaluation and therefore favours the more expensive configuration: reservation-table-aware dispatching relative to rule-based dispatching is 2.90% ($p = 0.0269$); under equal wall-clock time, before the speed-ups, the same comparison gives only 0.03% ($p = 0.8799$), close to zero. The equal-generation convention asks whether the extra routing computation improves the ranking in each generation; the equal-wall-clock convention asks whether makespan falls within the same time. Both questions are meaningful, but neither can stand in for the other.

The cost law. Reservation-table-aware dispatching raises the routing calls per transport task from 2 to at most $2N_A$, and the cost grows linearly with fleet size. The bound used by pruning is tighter when contention is weaker, and more candidates are skipped; the larger the fleet and the stronger the contention, the smaller the cut. The equivalence self-check of Section 4.5 agrees with this direction: of six instances the highest cut rate is on the least congested one (61.2%) and the lowest on the largest fleet (42.2%), i.e. pruning weakens but does not fail. Together the two trends *predict* that when the fleet is large enough and contention strong enough, the growth of trial-query cost will offset the gain.

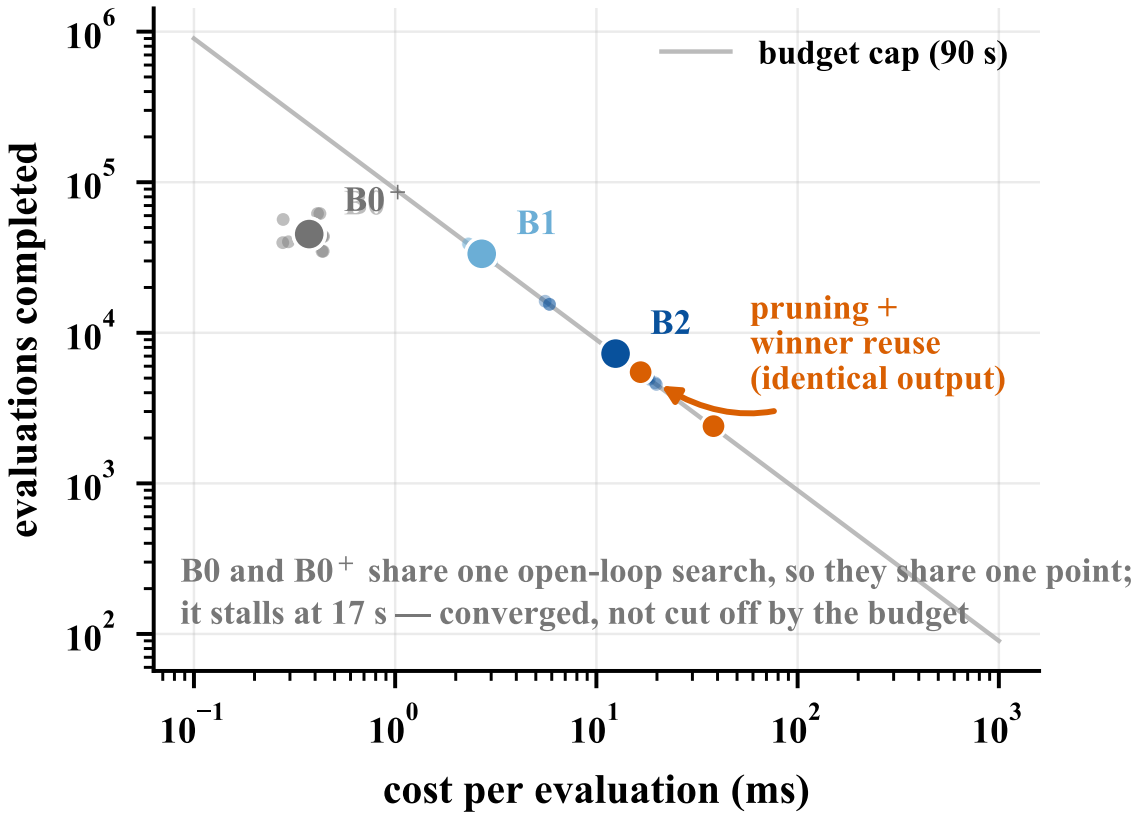
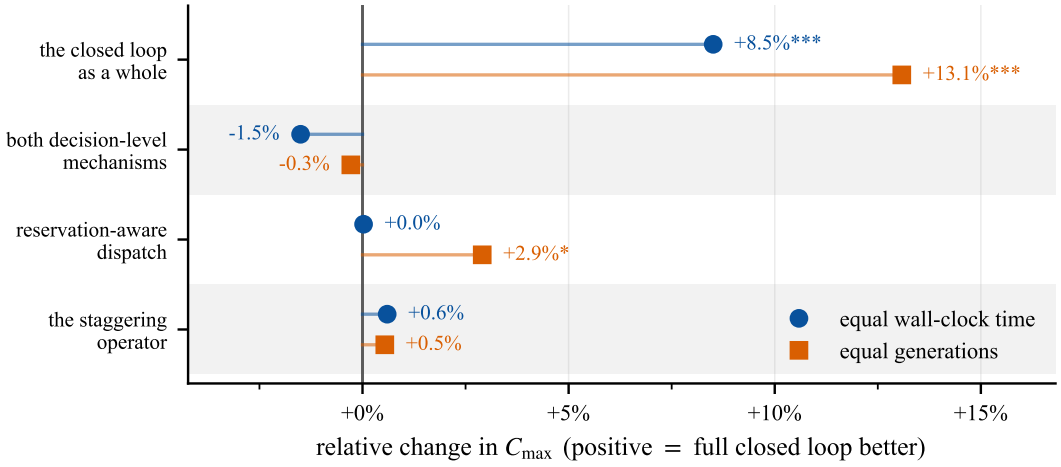


Fig. 6. A fixed wall-clock budget places the cost of a single evaluation and the number of evaluations completed on a hyperbola; the grey line is the 90-second cap shared by every configuration. A point on the line means cost × count equals that configuration's wall-clock time. The two closed-loop configurations lie on the line, i.e. every run uses the full budget; the open loop lies *below* it, because early stopping ends it after a little over four hundred generations without improvement, converging in about 17 seconds rather than being cut off by the budget (Section 5.1.2). B0 and B0$^+$ share the same open-loop search and therefore share a point; they differ only in how they are executed. Pale small points are per-instance measurements on three instances; the filled large point is their geometric mean, the only mean that preserves "cost × count = that configuration's wall-clock time" on a log axis. Orange markers are the off and on settings of pruning and winner-path reuse (the callout marks the on setting); by Proposition 3 the two produce identical output, so the displacement along the line reflects only the compute saved.

On the four vehicle-to-arm-ratio levels of Section 5.5 the sign alternates and none is significant, so this upper bound is not confirmed. The cost law itself (cost linear in N_A, pruning effect falling with N_A) is measured directly; the upper bound derived from it has not been verified.

Turning the two speed-ups on and off. The speed-ups and the dispatching rule were introduced one after the other, so placing two batches side by side cannot exclude differences of instance and baseline. We therefore run a separate control that changes only the speed-ups: same instances, same seeds, same budget, with pruning and winner-path reuse turned on or off.

Proposition 3 guarantees that the two optimizations *do not change the output.* On each of six instances six random chromosomes are taken; the makespans and dispatch sequences of the two settings are *identical bit for bit* in all 36



4 instances, 10 seeds, paired Wilcoxon; * p<0.05, ** p<0.01, *** p<0.001. Batch measured before the cost reduction of Sec. 4.5.

Fig. 7. The same set of comparisons under two budget conventions. Each row is named by the mechanism it isolates. The difference between the two markers comes from the convention: reservation-table-aware dispatching is 2.90% at an equal number of generations ($p = 0.0269$) and 0.03% at equal wall-clock time ($p = 0.8799$), close to zero. The batch ran *before* the two speed-ups of Section 4.5, which motivates the separate on/off control of the speed-ups below.

Table 8. Changing only pruning and winner-path reuse. "Off"/"on" means whether the two are enabled; the bold header is the setting adopted in this paper. The two settings produce bit-identical output (verified by the equivalence self-check), so the difference comes only from the compute saved. Same instances, same seeds, same wall-clock time; $n = 60$ pairs, two-sided Wilcoxon signed-rank test ($^{***}p < 0.001$); the aggregate row lists wins/losses/ties and p. Both ratio columns have the *off* setting as denominator, in opposite directions: evaluations as on/off, time per evaluation as off/on.

		C_{max} (time units)			Eval. count ratio	Speedup per eval.
Instance	Contention	Off	**On**	ΔC_{max}	(on/off)	(off/on)
A funnel	31.9%	44.80	42.10	−6.03%	2.21×	2.23×
A high	30.9%	41.60	37.70	−9.37%	2.40×	2.41×
A low	11.8%	60.50	57.00	−5.79%	2.79×	2.81×
B NA/NM 0.5	15.8%	66.70	65.10	−2.40%	2.11×	2.11×
B NA/NM 1.0	30.2%	51.90	48.90	−5.78%	2.31×	2.31×
B NA/NM 2.0	32.4%	44.50	42.20	−5.17%	1.98×	2.01×
Pooled (60 pairs)	—	—	—	−5.60%***		50/0/10, $< 10^{-4}$
HL pseudomedian and 95% CI: −4.72 [−6.34, −3.55] %						

cases; routing calls fall by 47.4% in total. Since the output is identical, turning the optimizations off only slows a single evaluation, and the makespan difference under equal wall-clock time comes *only* from the compute saved.

Table 8: off→on reduces makespan by a further 5.60% (50 wins / 0 losses, $p < 10^{-4}$), the number of evaluations inside the same budget is 2.23 times, and a single evaluation is sped up by 2.33. This control links the two batches: the same rule is -0.74% on the pre-speed-up matrix batch (significant on 0 of the 16 instances) and significantly positive on the post-speed-up ladder batch, while the speed-ups themselves change no selection.

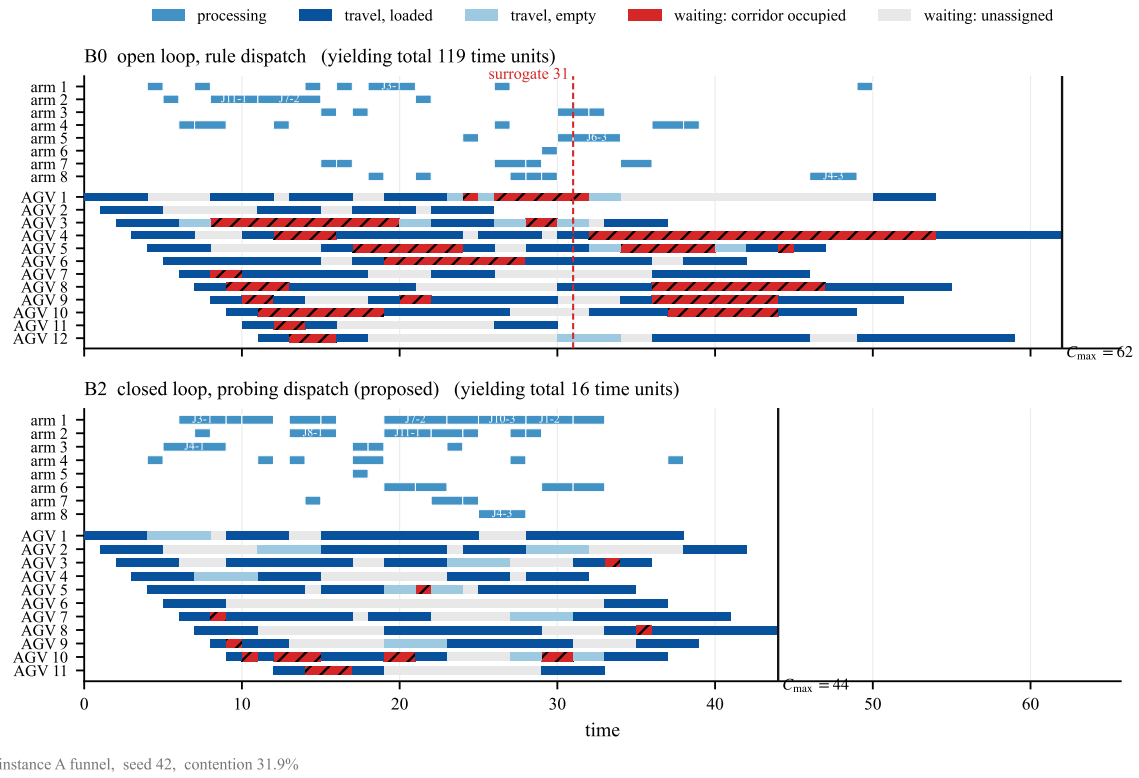


Fig. 8. B0 (top) and B2 (bottom) on the same funnel instance. Robotic arms and vehicles share the time axis; vehicle rows distinguish empty and loaded legs, and two kinds of gap. A gap between the two legs of the same transport task: the vehicle waits at a berth because a corridor is occupied, i.e. yielding wait, the delay treated in this paper. A gap between different tasks: the vehicle is idle. The two are coloured differently so that idle time is not mistaken for yielding wait.

5.8 Case study: Gantt chart and critical chain on the funnel instance

Aggregate numbers do not show what happens inside one solve. This section places B0 and B2 side by side on a single funnel instance and colours the links of the critical chain by Table 3.

Figure 8 draws robotic arms and vehicles on a shared time axis; vehicle rows distinguish empty and loaded legs, and yielding wait is coloured separately. Three observations follow. In the open-loop solution, yielding wait appears in stretches on the corridors near the load/unload point and concentrates on a few vehicles. The closed-loop solution greatly reduces these waits, and several operations are assigned to an arm that is not the fastest, i.e. assignments of the kind described by (14), trading a longer processing time for a shorter path with better traffic conditions. The closed-loop solution also balances vehicle load more evenly, a consequence of the first two points rather than a separate mechanism.

Figure 9 colours each link by the five labels of Table 3. On B0's chain `corridor` links take a substantial share; on B2 they largely disappear and are replaced by `machine` and `upstream`. This is the dilution described in Section 5.6.3: once one corridor delay is removed, another constraint becomes tight.

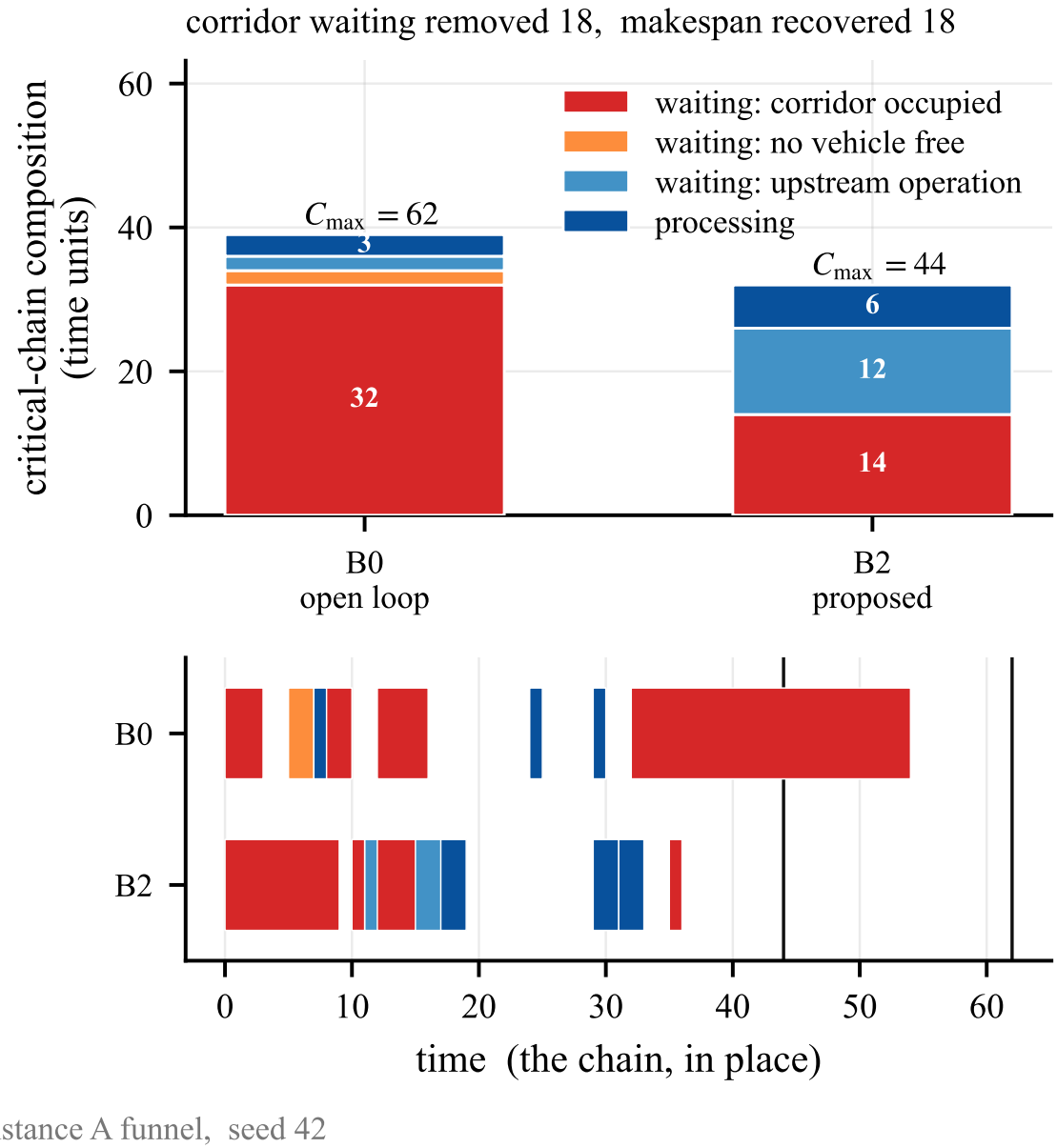


Fig. 9. Links of the critical chain coloured by Table 3. Top: composition of the two critical chains; the title places "corridor wait removed" beside "makespan actually reduced", and the difference is the part absorbed by the newly tight constraints. Bottom: the same chain in position on the time axis, showing that the bars above decompose one chain rather than forming a histogram of unrelated events. In the closed-loop solution `corridor` largely disappears and is replaced by arm and upstream links.

5.9 Degenerate comparison on public benchmarks

The preceding sections compared different configurations of the same codebase on self-generated instances, which excludes implementation differences among configurations but cannot calibrate the absolute level of the implementation: even if the upper-level search is weak, the differences among configurations remain credible, but the absolute makespan reflects only the implementation. This section calibrates the upper-level search on public benchmarks.

What is compared. After the exclusivity constraint (8) is removed, the model of this paper degenerates exactly to standard FJSP_PT (Section 3), the *same mathematical problem* as the public benchmarks. $\delta_{\text{ret}} = 0$ is additionally taken to align the makespan definition of this batch; it is already a model parameter (Table 2), not the removal of a further constraint. After the degeneration there is no corridor contention, neither mechanism enters, and what is tested is the *upper-level search*, not the framework of this paper. The results are reported in a table separate from the self-generated instances, so that the two conclusions do not share one table.

Instances and protocol. The 20 instances compiled by Homayouni and Fontes [7] are taken (originating in Fattahi et al. [2], 2–12 jobs, 4–48 operations, 2–8 machines), with two vehicles as in the literature [11] and no return of finished goods. Travel times are used *as published*, without a shortest-path synthesis: Section 5.1 already noted that 2 of the matrices are not the shortest-path closure of any graph, and a synthesis would change the instance. After degeneration the road network G enters the model only through t^*, so the published matrix can be used directly as t^* without requiring a consistent G. Contention requires G itself, which this batch does not provide, so the batch can be used *only* in the degenerate setting. The published optima of two small instances (SFJST01, SFJST03) were checked by hand, confirming that neither includes a return trip. Reference values: 16 instances have a MILP-proven optimum [7]; the remaining 4 take the published best-known [11].

Results. Table 9 is 20 instances × 10 seeds. All 200 runs pass three checks: no solution below the composite lower bound, no validation failure, and, on the 16 instances with a proven optimum (160 runs), no run below that value; since the degenerate setting and the literature pose the same problem, this confirms that conversion and conventions are correct. On 12/20 instances the best solution is *identical bit for bit* to the reference, including all 10 small instances (SFJST01–10); the mean gap over the 20 instances is 2.68% (median 0.00%). The gap rises with size, but not monotonically instance by instance: MFJST03 and MFJST06 both beat their respective smaller neighbours. The four largest instances (32–48 operations) have mean 9.96%, worst 12.93%; none of the 16 instances with at most 24 operations exceeds 5.45%.

The gap stems mainly from search capability, not from an insufficient budget. Of the literature's 16 proven optima, the MILP on the largest instance took nearly ten hours (MFJST06, 34 796 seconds); each run of this paper uses a few seconds. The budget is therefore raised by one step (generation cap five times, early-stop threshold 3.3 times, measured wall-clock time to 2.5 times) and re-run on the 10 MFJST instances in the last ten rows of Table 9—the gap concentrates there. The mean gap of these ten falls from 5.37% to 4.53% (the base contains only these ten, a different convention from the 2.68% over the 20 instances above), i.e. 2.5 times the compute yields an improvement of only 0.84 percentage points; 5 of them show no decrease, and the instance with the largest gap falls only from 12.93% to 12.85%. Under both budgets, *none* of the 300 runs reaches the generation cap; all are terminated by early stopping, so the larger budget did not lengthen the search. The gap therefore mainly reflects the capability limit of the upper-level search. Narrowing it on large instances is item (3) of Section 6.3.

Table 9. Degenerate comparison on public benchmarks (exclusivity constraint removed; the same problem as the literature). In the "reference" column, OPT is a MILP-proven optimum and BK is a published best-known; the gap is computed from the best solution over 10 seeds. An asterisk marks instances whose best solution is identical bit for bit to the reference. Data: `experiments_database/gap_ideal.csv`.

			$C_{\max}$ (time units)			
Instance	Ops	Ref.	Value	Best	Mean	Gap
SFJST01	4	OPT	70	70*	70.0	0.00%
SFJST02	4	OPT	111	111*	111.0	0.00%
SFJST03	6	OPT	223	223*	223.0	0.00%
SFJST04	6	OPT	359	359*	359.0	0.00%
SFJST05	6	OPT	123	123*	123.0	0.00%
SFJST06	9	OPT	324	324*	324.0	0.00%
SFJST07	9	OPT	409	409*	409.0	0.00%
SFJST08	9	OPT	269	269*	269.5	0.00%
SFJST09	9	OPT	220	220*	222.0	0.00%
SFJST10	12	OPT	531	531*	531.0	0.00%
MFJST01	15	OPT	485	485*	485.0	0.00%
MFJST02	15	OPT	463	476	483.4	2.81%
MFJST03	18	OPT	482	482*	504.7	0.00%
MFJST04	21	OPT	576	586	623.0	1.74%
MFJST05	21	OPT	532	561	592.8	5.45%
MFJST06	24	OPT	652	677	728.3	3.83%
MFJST07	32	BK	898	965	1023.3	7.46%
MFJST08	36	BK	900	970	1027.0	7.78%
MFJST09	44	BK	1098	1226	1279.2	11.66%
MFJST10	48	BK	1230	1389	1473.0	12.93%

Sources of the gap between the composite lower bound and the best solution. The median gap between the composite lower bound and the best solution obtained is 42%. It has two components: how far the bound lies below the true optimum, and how far the search solution lies above it. On the self-generated instances the optimum is unknown and the two components cannot be separated; on the public instances they can. Only the 16 instances with a proven optimum are taken: the reference of the other 4 is a published best-known, the true optimum can only be smaller, and using it as a denominator would overstate "how much the bound lies below the optimum". On these 16 instances, the same composite lower bound lies a median 10.3% below the proven optimum and a median 10.8% below the best solution of this paper; taking the difference instance by instance, the search solution lies a median 0.0 percentage points above the optimum (most instances already hit the optimum), and at most 4.1 percentage points. Thus on this *contention-free* batch, almost all of the gap between the bound and the best solution comes from slack in the bound, not from the search missing the optimum. This proportion cannot be transferred directly to the self-generated instances. The composite lower bound contains no corridor contention; contention can only raise or keep the true optimum of a self-generated instance while the bound stays the same, so the distance from the bound to the true optimum is no smaller than on the public instances. What can be established is that the 42% on the self-generated instances *cannot* be read as a search gap, because the same bound already lies 10.3% below the optimum without contention. The distance between the search and the optimum on the self-generated instances remains unknown (limitation (3) of Section 6.2).

6 Conclusion

This paper studies scheduling in which flexible assignment and conflict-free AGV routing must be decided jointly: travel time is produced by routing, and when the corridor to a fast arm is congested, a slower arm with better traffic conditions may be the better assignment. Existing methods either schedule on a constant matrix or freeze the assignment before resolving conflicts, so yielding wait does not enter the evaluation of candidates in the search. To address this gap, our bilevel framework connects the two levels by an evaluation path and a decision path and measures their contributions separately under the same wall-clock budget: the fitness of each candidate is the makespan after conflict-free routing, and vehicles are selected by a trial query on the reservation table rather than by ideal travel time. Because a conflict-free evaluation costs more than an order of magnitude more than a matrix lookup, we also give speed-ups that change neither the selected vehicle nor the timetable, and test whether extending the inter-layer interface beyond the two paths (a surcharge on corridor–time slots, or reassignment and staggering from the yield record) can further reduce makespan under the same budget.

6.1 Findings and contributions

(1) A controllable characterization of the reassignment trade-off. When machine flexibility, heterogeneous processing efficiency, and corridor contention are all present and the schedule targets the makespan with yielding wait counted, a slower arm with better connectivity may be the better assignment if the corridor to a fast arm is congested. On top of the problem setting of [21], we construct instance families with controllable contention intensity to examine how high contention must be before counting yielding wait in the schedule yields a gain.

(2) A closed-loop bilevel framework with routing-in-the-loop evaluation. The upper level searches assignment and sequencing with a memetic algorithm, and the lower level performs conflict-free routing by time-window reservation; the fitness of each candidate is the makespan after the lower level has completed every transport. Proposition 2 and Lemma 1 guarantee that every chromosome is decodable and that the evaluation is consistent with the conflict model of Section 3, with no repair operator needed. Relative to the open loop, the closed loop reduces makespan by 18.86% under rule-based dispatching ($p < 10^{-4}$), and by 17.75% under reservation-table-aware dispatching ($p < 10^{-4}$).

(3) Reservation-table-aware dispatching and two output-preserving speed-ups. Before a vehicle is selected, both transport legs are trial-queried on the reservation table, and the earliest feasible delivery replaces the unimpeded-shortest-path estimate; within the closed loop this reduces makespan by a further 2.45% ($p = 0.0054$). The rule raises the cost per evaluation to 12.2 times that of rule-based dispatching, enough to offset its gain under the same wall-clock budget; admissible lower-bound pruning and winner-path reuse change neither the selected vehicle nor the timetable (Proposition 3, with a bitwise self-check on six instances), cut router calls by 47.4%, and lower the cost multiple to 5.23.

(4) The two paths differ in magnitude, are near-additive, and are not substitutes for each other. The closed loop remains significant after Holm correction on 9 of the 10 instances; reservation-table-aware dispatching is significant when pooled under both the open and the closed loop (4.59%, $p < 10^{-4}$; 2.45%, $p = 0.0054$, 95% interval $[0.94\%,\ 5.19\%]$), but on no instance individually. The end-to-end 21.35% is close to the prediction under independent composition, 22.6% (near-additive, no positive interaction detected). The schedule-then-resolve route can obtain only the open-loop dispatching gain of 4.59%; because its first-stage search ignores yielding wait, the reduction from closed-loop evaluation (18.86%) is unavailable to it.

6.2 Limitations

(1) Scope of the instance families, and why self-generated instances are used. The main experiment fixes the size and changes only structure (Section 3.7), so we draw no conclusion about how the mechanisms vary with size. Heterogeneity H is fixed at 0.3; the scan of four H levels comes from the smaller independent instances of Section 5.5, so "the gain does not rise with H" holds only at *that* size. The layout family is spliced from two topologies: the three dumbbell levels change only network capacity, while the two grid levels change topology and distance together (the farthest leg rises from 10.0 to 18.0, Section 5.1). The contention intensity that characterizes the family is also only a one-dimensional measure and does not separate the part of yielding wait that reassignment can bypass. Hence only the differences among the three dumbbell levels can be attributed to capacity alone, and the five levels should not be joined into one curve by contention intensity; these two points are also possible reasons why gain and contention intensity are not in strictly the same order. Self-generated instances are used because this direction has no usable public benchmark: common FJSP_PT benchmarks publish constant matrices, and in the set we examined 7/7 matrices are asymmetric and 2 are not the shortest-path closure of any graph, so contention cannot be defined (Section 5.1); the two works with an explicit road network use different reservation units and cannot be compared directly (Section 3.5).

(2) The makespan gain from dispatching is positive on average, but the class of instances on which it is stable cannot yet be determined. 10 seeds per instance suffice to resolve the closed-loop effect (9/10 instances significant after Holm correction). The effect of reservation-table-aware dispatching is only a few percentage points, of the same order as the dispersion among seeds: 2/10 instances are significant before correction and 0/10 after. The gain can therefore be confirmed as positive only on the 100 pooled pairs (95% interval [0.94%, 5.19%], lower end less than one percentage point from zero). Raising the number of seeds by about an order of magnitude would be needed to determine the class of instances on which it appears and to test the fleet-size upper bound predicted by the cost law of Section 5.7.

(3) The upper-level search is calibrated on contention-free public benchmarks; with corridor contention, the distance of solutions from the optimum cannot yet be determined. The configurations share one code base and report differences among levels; the strength of the upper-level search is common to all levels and does not affect those differences. The degenerate comparison of Section 5.9 is bitwise identical to the literature on 12/20 instances, and all 10 small instances reach a proven optimum. The gap rises with size: the four largest instances average 9.96%, and 2.5 times the compute improves it by only 0.84 percentage points, so the gap reflects a capability limit rather than an insufficient budget. On the 16 public instances with a proven optimum, the search solution is at most 4.1 percentage points from the optimum, and the bound gap comes almost entirely from slack in the bound. On the self-generated instances the composite lower bound contains no contention, and its median gap 42% to the best solution cannot be read as a search gap; we draw no quantitative conclusion about that distance. The public instances contain no corridor contention, and lower-level routing with the reservation table has no proven optimum to compare against; a secondary effect of a few percentage points (such as 2.45%) may be of the same order as the distance of the implementation from the optimum.

(4) The surcharge and yield-record-guided local search did not further reduce makespan; if their evaluation cost is reduced separately, they must be remeasured. Both were assessed in the same code base under the same wall-clock budget, so what was measured is the trade-off between cost per evaluation and number of evaluations: speeding up all configurations by the same factor leaves that trade-off and the sign of the result unchanged, and only reducing the cost of one interface separately could change the sign, as the dispatching speed-ups of Section 4.5 illustrate.

Hence, if neighbourhood evaluation is changed to re-solve only the perturbed part, it must be measured again (item (4) of Section 6.3).

(5) Dynamic events such as robotic-arm and vehicle failures are not considered. Assumption A7 excludes robotic-arm and vehicle failures and dynamic job arrivals from the model. The reported results are limited to the static setting in which the job set and processing times are known over the planning horizon and equipment does not fail. With failures or online arrivals, the value of a mechanism also includes how fast it repairs a schedule, not only the quality of the schedule it constructs; this aspect is not measured here (item (5) of Section 6.3).

6.3 Future work

(1) Remeasure on a larger explicit road network, and separate contention from chain substitutability. This corresponds to limitation (1). The main experiment fixes the size, so the first step is to test on a larger explicit corridor network whether the closed-loop and dispatching gains keep the same magnitude. Layout design should treat "whether congestion can be bypassed by reassignment" and "whether the critical chain has a single dominant bottleneck" as two independent factors; the funnel share φ failed to separate the two layout classes it was expected to separate (Section 3.7), so a measure first validated on instances with known answers is needed.

(2) Raise the number of seeds to determine the class of instances on which the dispatching gain is stable. This corresponds to limitation (2). The dispatching gain is confirmed to be positive on average. Raising the number of seeds by about an order of magnitude and sampling the vehicle-to-arm ratio more densely would determine where it is stable and test the cost law: whether there is a fleet size beyond which the growth of trial-query cost offsets the gain.

(3) Measure the distance of solutions from the optimum on instances with corridor contention. This corresponds to limitation (3). The 9.96% on the large public instances is insensitive to longer search, the seed-to-seed standard deviation on the four instances reaches 2.3%–4.5% of the mean, and all runs end by early stopping, so the encoding or neighbourhood needs improvement. At the small end, exact solutions by logic-based Benders decomposition [21] would separate, under corridor contention, the share of the gap due to bound slack from that due to the search, and would test whether the surcharge and yield-record-guided local search still fail to reduce makespan under a stronger search. A stronger search does not affect the differences among levels, but may help determine the class of instances on which the dispatching gain is stable (limitation (2)).

(4) Reduce cost by incremental evaluation, then remeasure yield-record-guided local search. This corresponds to limitation (4). Decoder state advances in one direction, so when a perturbation affects only the sequence after some position, earlier events and reservations are unchanged. Re-solving only the perturbed part separately reduces the cost per evaluation of this local search, which can then be remeasured under the same wall-clock budget.

(5) Measure the speed of repairing the schedule under robotic-arm and vehicle failures. This corresponds to limitation (5). Work on the dynamic flexible job shop has already brought machine failures, urgent insertions, and job arrivals into rescheduling [1, 6, 23]. The value of a mechanism then also includes the speed of repairing a schedule, not only the time and quality of constructing one. Whether an evaluator that re-solves from scratch or a method that modifies only a local neighbourhood is preferable cannot be inferred from the static setting.

References

[1] Stéphane Dauzère-Pérès, Junwen Ding, Liji Shen, and Karim Tamssaouet. 2024. The Flexible Job Shop Scheduling Problem: A Review. *European Journal of Operational Research* 314, 2 (2024), 409–432. doi:10.1016/j.ejor.2023.05.017

[2] Parviz Fattahi, Mohammad Saidi Mehrabad, and Fariborz Jolai. 2007. Mathematical Modeling and Heuristic Approaches to Flexible Job Shop Scheduling Problems. *Journal of Intelligent Manufacturing* 18, 3 (2007), 331–342. doi:10.1007/s10845-007-0026-8

[3] Giuseppe Fragapane, René de Koster, Fabio Sgarbossa, and Jan Ola Strandhagen. 2021. Planning and Control of Autonomous Mobile Robots for Intralogistics: Literature Review and Research Agenda. *European Journal of Operational Research* 294, 2 (2021), 405–426. doi:10.1016/j.ejor.2021.01.019

[4] Jie Fu, Bin Yang, Zhixing Chang, Yuanrong Zhang, Jiarui Wang, Xiaotong Wang, and Lei Wang. 2025. Integrated Production–Logistics Scheduling in Flexible Assembly Shops Using an Improved Genetic Algorithm. *Machines* 13, 12 (2025), 1090.

[5] Jianqi Gao, Yanjie Li, Xinyi Li, Kejian Yan, Ke Lin, and Xinyu Wu. 2023. A Review of Graph-Based Multi-Agent Pathfinding Solvers: From Classical to Beyond Classical. *Knowledge-Based Systems* 283 (2023), 111121. doi:10.1016/j.knosys.2023.111121

[6] Yong Gui, Dunbing Tang, Haihua Zhu, Yi Zhang, and Zequn Zhang. 2023. Dynamic Scheduling for Flexible Job Shop Using a Deep Reinforcement Learning Approach. *Computers & Industrial Engineering* 180 (2023), 109255. doi:10.1016/j.cie.2023.109255

[7] Seyed Mahdi Homayouni and Dalila B. M. M. Fontes. 2021. Production and Transport Scheduling in Flexible Job Shop Manufacturing Systems. *Journal of Global Optimization* 79, 2 (2021), 463–502. doi:10.1007/s10898-021-00992-6

[8] Kazumi Kasaura, Mai Nishimura, and Ryo Yonetani. 2022. Prioritized Safe Interval Path Planning for Multi-Agent Pathfinding with Continuous Time on 2D Roadmaps. *IEEE Robotics and Automation Letters* 7, 4 (2022), 10494–10501. doi:10.1109/LRA.2022.3187265

[9] Jeonghyeon Kim and Junwoo Kim. 2026. Congestion-Aware Scheduling for Large Fleets of AGVs Using Discrete Event Simulation. *Electronics* 15, 1 (2026), 139. doi:10.3390/electronics15010139

[10] Ruiqi Li, Jianlin Mao, Xing Wu, Wenna Zhou, Chengze Qian, and Haoshuang Du. 2026. A New Framework for Job Shop Integrated Scheduling and Vehicle Path Planning Problem. *Sensors* 26, 2 (2026), 543. doi:10.3390/s26020543

[11] Che Han Lim and Seung Ki Moon. 2023. A Two-Phase Iterative Mathematical Programming-Based Heuristic for a Flexible Job Shop Scheduling Problem with Transportation. *Applied Sciences* 13, 8 (2023), 5215. doi:10.3390/app13085215

[12] Wenhao Lu, Shuai Guo, Tao Song, and Yuwen Li. 2022. Analysis of Multi-AGVs Management System and Key Issues: A Review. *Computer Modeling in Engineering & Sciences* 131, 3 (2022), 1197–1227. doi:10.32604/cmes.2022.019770

[13] Xiangfei Lyu, Yuchuan Song, Changzheng He, Qi Lei, and Weifei Guo. 2019. Approach to Integrated Scheduling Problems Considering Optimal Number of Automated Guided Vehicles and Conflict-Free Routing in Flexible Manufacturing Systems. *IEEE Access* 7 (2019), 74909–74924. doi:10.1109/ACCESS.2019.2919109

[14] Leilei Meng, Weiyao Cheng, Chaoyong Zhang, Kaizhou Gao, Biao Zhang, and Yaping Ren. 2025. Novel CP Models and CP-Assisted Meta-Heuristic Algorithm for Flexible Job Shop Scheduling Benchmark Problem with Multi-AGV. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 55, 11 (2025), 8455–8468. doi:10.1109/TSMC.2025.3604355

[15] Jacqueline Otala, Alden Minard, Golshan Madraki, and Seyedamirabbas Mousavian. 2021. Graph-based modeling in shop scheduling problems: Review and extensions. *Applied Sciences* 11, 11 (2021), 4741. doi:10.3390/app11114741

[16] Haoxiang Qin, Yi Xiang, Yuyan Han, Yuting Wang, Junqing Li, and Quanke Pan. 2026. A Knowledge Region Selection Enhanced Quality-Diversity Algorithm for Real-World Flexible Job Shop Scheduling with Automated Guided Vehicles Transportation. *Engineering Applications of Artificial Intelligence* 164 (2026), 113352. doi:10.1016/j.engappai.2025.113352

[17] Behnam Rahimikelarijani, Hamed Fazlollahtabar, and Sina Nayeri. 2020. Multi-Objective Multi-Load Tandem Autonomous Guided Vehicle for Robust Workload Balance and Material Handling Optimization. *SN Applied Sciences* 2 (2020), 1032. doi:10.1007/s42452-020-3002-y

[18] Yong Jin Suh and Jin Young Choi. 2022. Efficient Fab Facility Layout with Spine Structure Using Genetic Algorithm under Various Material-Handling Considerations. *International Journal of Production Research* 60, 9 (2022), 2816–2829. doi:10.1080/00207543.2021.1904159

[19] Jiachen Sun, Zifeng Xu, Zhenhao Yan, Lilan Liu, and Yixiang Zhang. 2023. An Approach to Integrated Scheduling of Flexible Job-Shop Considering Conflict-Free Routing Problems. *Sensors* 23, 9 (2023), 4526. doi:10.3390/s23094526

[20] Karim Tamssaouet and Stéphane Dauzère-Pérès. 2023. A General Efficient Neighborhood Structure Framework for the Job-Shop and Flexible Job-Shop Scheduling Problems. *European Journal of Operational Research* 311, 2 (2023), 455–471. doi:10.1016/j.ejor.2023.05.018

[21] Roel W. M. van Os and Twan Basten. 2026. An Exact Decomposition-Based Approach to the Conflict-Free-Transportation-Constrained Flexible Job-Shop Scheduling Problem. *Computers & Operations Research* 187 (2026), 107342. doi:10.1016/j.cor.2025.107342

[22] Bin Xin, Sai Lu, Qing Wang, and Fang Deng. 2025. A Review of Flexible Job Shop Scheduling Problems Considering Transportation Vehicles. *Frontiers of Information Technology & Electronic Engineering* 26, 3 (2025), 332–353. doi:10.1631/FITEE.2300795

[23] Hao Xiong, Shiqing Shi, Dongxu Ren, and Jialong Hu. 2022. Dynamic Scheduling for Flexible Job Shop with New Job Insertions by Deep Reinforcement Learning. *Applied Soft Computing* 127 (2022), 109360. doi:10.1016/j.asoc.2022.109360

[24] Fayong Zhang, Rui Li, and Wenyin Gong. 2024. Deep Reinforcement Learning-Based Memetic Algorithm for Energy-Aware Flexible Job Shop Scheduling with Multi-AGV. *Computers & Industrial Engineering* 189 (2024), 109917. doi:10.1016/j.cie.2024.109917

[25] Yulun Zhang, He Jiang, Varun Bhatt, Stefanos Nikolaidis, and Jiaoyang Li. 2024. Guidance Graph Optimization for Lifelong Multi-Agent Path Finding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*.

Received XX XX 2026; revised XX XX 2026; accepted XX XX 2026